《在 Smalltalk-80 中运用模型—视图—控制器用户界面范式的实践指导》中文译本

原题：A Cookbook for Using the Model-View-Controller User Interface Paradigm in Smalltalk-80

作者：Glenn E. Krasner、Stephen T. Pope

出处：Journal of Object-Oriented Programming (JOOP)，1988 年 8–9 月

译者说明：这是 1988 年的 Smalltalk-80 实现论文，不应与 Reenskaug 于 1979 年提出的初始 MVC 设计混为一起。译文按原文结构翻译。代码、类名、消息选择子、图号和参考文献保持原样。原论文扫描页附在译文后，便于查看未重绘的图表及代码。

引言（Introduction）

Smalltalk-80 编程环境的用户界面（见 [Goldberg 83]）采用了一种特定的信息表示、显示与控制方法。之所以采用这种方法，是为了实现两个目标：（1）建立支持高度交互式软件开发过程所需的一组特殊系统组件；（2）提供一组通用系统组件，使程序员能够容易创建可移用的交互式图形应用程序。

本文假定读者对 Smalltalk-80 语言和编程环境已有基本了解。不熟悉这些内容的读者，可以参阅 [Goldberg and Robson 83] 和 [Goldberg 83]，其中包含入门和教程材料。

MVC 与可复用性、可插接性问题（MVC and the Issues of Reusability and Pluggability）

和开发其他程序一样，构建交互式应用时，组件的模块化具有很大的价值。尽可能隔离不同的功能单元，能让应用设计者在不必了解其他单元全部细节的前提下，理解并修改某个单元。

我们在 Smalltalk-76 系统中的经验表明，一种特别的模块化方式——把应用组件分成三部分——带来的收益，不只是让设计者的工作更容易。它把以下三件事分开：（1）表示应用问题领域基础模型的部分；（2）把模型呈现给用户的方式；（3）用户与模型交互的方式。

模型—视图—控制器（Model–View–Controller，MVC）编程就是这种三分法的应用。不同类的对象分别承担：与应用领域有关的操作（Model）；显示应用状态（View）；处理用户与模型及视图之间的交互（Controller）。

在早期 Smalltalk 系统的用户界面中，工具通常由四种基本显示形式组合而成：文本段落、文本列表（选项表）、选择“按键”以及图形（位图或像素图）。这些工具也通常采用三种基本交互模式：浏览（Browsing）、检查（Inspecting）和修改（Editing）。

Smalltalk-80 的目标之一，就是只定义一次处理这些显示形式和交互模式的界面组件，然后通过 MVC 方法，让所有编程环境工具以及用户自行编写的应用共用这些组件。

我们还设想，MVC 方法允许程序员先定义新的类，让这些类承载特定应用领域的信息，从而构建应用模型；随后通过把预先定义好的用户界面类的实例“插接”到一个复合视图（窗口）中，完成界面设计。

这种可插接性（Pluggability）不仅适用于显示形式，也适用于控制（修改）模式。尽管在交互式应用中，显示方式和交互方法显然有关联，但将二者的功能分开仍然有优势。

使用浮动选项表还是固定选项表、赋给键盘键与Mouse功能键什么含义、如何调度多个视图，这些都应当能够独立于模型及其视图进行选择。在适当情况下，这些选择甚至可以留给最终用户。

模型—视图—控制器构想（The Model-View-Controller Metaphor）

为了解决上述问题，我们发展出了 MVC 构想，以及围绕它思考并实现交互式应用的结构化方法。

模型是系统应用中真正完成工作的组件，也就是对应用领域进行模拟的部分。模型与视图明确分开；视图负责显示模型的某些方面。控制器向模型发送消息，并在模型、相关视图与交互式输入设备（如键盘、Mouse）之间提供接口。

可以认为，一个视图与一个控制器紧密关联；每个视图或控制器都恰好对应一个模型，但一个模型可以对应许多组视图和控制器。

模型（Models）

应用的模型，是对应用核心结构的一种领域专用软件模拟或实现。它可以简单到只是一个整数（例如计数器的模型）或字符组（例如文本Editor的模型）；也可以是复杂对象，例如 Smalltalk-80 集合类或其他组合类的子类实例。本文后面的章节会讨论多个模型实例。

视图（Views）

在这一构想中，视图负责所有图形方面的工作：它向模型请求数据，然后显示数据。视图除了包含绘制所需的组件，还可以包含子视图，也可以被包含在上层视图中。

上层视图提供不同层级之间的图形变换、窗口管理和裁剪能力。显示消息通常从顶层视图（应用窗口的标准系统视图）向下传给各个子视图。

控制器（Controllers）

控制器负责在相关模型、视图与输入设备（如键盘、指针设备、计时器）之间建立接口。控制器也处理与其他视图—控制器对之间的交互调度：它追踪Mouse在不同应用视图之间的移动，并实现处理Mouse按键活动和输入传感器数据的消息。

虽然选项表也可以被看成视图—控制器对，但通常更适合把它们视为输入设备的一部分，因此归入控制器的职责范围。

广播变更（Broadcasting Change）

在上述方案中，视图和控制器各自恰好有一个模型，而一个模型可以关联一个或多个视图和控制器。为了最大限度地封装数据并实现代码复用，视图和控制器必须明确知道自己的模型，但模型不应当知道它有哪些视图和控制器。

模型的变化通常始于控制器把用户操作转换为发给模型的消息。这种变化应当反映在模型的所有视图中，而不只是触发变更的控制器所关联的视图。

依赖对象（Dependents）

为了管理变更通知，我们引入了把对象视为依赖对象（Dependent）的概念。模型的视图和控制器被加入到模型的一个列表中，作为模型的依赖对象；当模型的某个方面发生变化时，它们会得到通知。

模型发生变化后，会广播一条消息来通知所有依赖对象。通知消息可以带参数，因此模型可以发出多种类型的变更消息。每个视图或控制器以适当方式响应自己关心的模型变化。

图 1：MVC 的状态和消息转交。（参见原文扫描图。）

标准交互循环（A Standard for the Interaction Cycle）

因此，在 MVC 构想下，标准的交互循环是：用户执行某个输入操作；当前活动控制器通知模型，让模型相应地改变自己。模型执行规定的操作，可能改变自身状态，然后向依赖对象（视图和控制器）广播自己已经改变的消息，并且可能说明变化的性质。

随后，视图可以查找模型的新状态，并在必要时更新显示；控制器也可能根据模型的新状态改变交互方式。图 1 用示意图展示了这个消息转交过程。

MVC 的一种实现（An Implementation of Model-View-Controller）

Smalltalk-80 对 MVC 构想的实现，由 `Model`、`View`、`Controller` 三个抽象上层类及大量具体子类组成。抽象类包含 MVC 三部分通用的状态与行为；具体类包含 Smalltalk-80 中应用功能和用户界面组件所需的专门状态与行为。

由于我们首先需要的是支持软件开发工具的一组界面组件，因此 `Model`、`View` 和 `Controller` 最基本的具体子类，主要处理可调度视图、文本、文本列表、选项表、图形、图标等内容。

Model 类（Class Model）

模型必须具备两种行为：保存依赖对象，以及向依赖对象广播变更消息。模型持有一个依赖对象集合。`Model` 类提供向该集合添加和移除依赖对象的消息共议，也提供广播变更通知的能力。

向模型发送 `changed` 消息，会使模型向每个依赖对象发送 `update` 消息。向模型发送 `changed: aParameter`，会使每个依赖对象收到相应的 `update: aParameter` 消息。

一个简单但又颇有代表性的 MVC 例子，是 [Goldberg and Robson 83] 中的 `FinancialHistoryView` 教程。图 2 展示了收支记录 `FinancialHistory` 的界面，本文末尾的 MVC 实现示例将进一步讨论它。

这个例子创建了一个条形图视图，让它成为带文字标记数值字典的依赖对象（例如，“rent（住房支出）—500 美元”）。复合的 `FinancialHistoryView` 含有两个子视图，也就是两张条形图。它们的模型是两个不同的字典，分别保存收入和支出。这两个字典又是中心模型（`FinancialHistory` 类实例）的实例变量。

图 2 还显示了 `FinancialHistoryController` 使用的浮动选项表，其中有 `spend`（支出）与 `receive`（收入）两个选项，以及一个输入提示器（`FillInTheBlank`），问问用户“要花多少金额”。选项表与提示器不会同时显示。

用户与这个收支应用交互时，例如可以从选项表中选择某一项，为rent（住房支出）添加一笔支出。控制器随后向模型（字典）发送消息，模型则向自己发送 `changed` 消息。结果，条形图收到 `update` 消息。条形图响应这一消息，重新从字典读取数值，并通过显示消息绘制更新后的条形图。图 3 是这一 MVC 交互的流程图。

带参数的变更消息（如 `self changed: someAspect`）用于告诉依赖对象模型的哪个方面发生变化，从而减少不必要的视图更新。

例如，前面保存收入与支出字典的模型 `householdFinances`，本来可以直接充当两张条形图的模型，并分别提供展示收入和支出的消息共议。这种情况下，`update` 消息可以带一个参数，说明变化发生在支出还是收入方面。

根据应用的不同，Smalltalk-80 的许多基本系统结构类都可以充当 MVC 模型。系统或用户应用里的视图，其底层模型可以是简单对象（数字或字符组）、集合类实例（`OrderedCollection`、字典、位图或 `DisplayText`），也可以是复杂组合对象（网状结构、数据库、事件列表、收支记录等）。

图 3：收支应用中，普通消息发送和依赖对象更新消息的流程。（参见原文扫描图。）

View 类（Class View）

抽象上层类 `View` 包含系统中所有视图的通用行为：与模型和控制器的交互、子视图与上层视图的交互、坐标变换，以及显示矩形的相关操作。`View` 的许多子类则实现不同的图形界面工具。

每个视图实例恰好有一个模型和一个控制器。模型通常被显式设置。由于视图类与控制器类经常配套设计，视图的控制器通常只需初始化为对应控制器类的一个实例。为此，许多视图子类定义了 `defaultControllerClass` 消息，用来返回合适的控制器类。

视图可以有零个或多个子视图，视图与其上层视图、子视图之间可以进行灵活的坐标变换。`View` 类实例保存指向上层视图的引用、一个可能为空的子视图集合，以及一个 `WindowingTransformation` 实例，用来表示视图坐标系与上层视图坐标系之间的变换（平移和缩放）。

此外，`View` 还提供添加和移除子视图，以及修改变换关系的默认消息共议，因此可以简单、灵活地把多个子视图拼成复合视图。

`View` 的第三类行为与绘制有关。由于假定具体视图子类会使用 `Form`、`Pen`、`Line` 或其他图形对象完成实际绘制，所以 `View` 只提供通用的显示行为。尤其是，`View` 并没有专门保存绘图对象的实例变量。它假定子视图结构的顶层会在某种载体上显示内容，通常是显示屏。

因此，视图会缓存从自己的内部坐标系到显示设备坐标系的变换，也就是本视图及所有上层视图变换的合成结果。这样可以更快地把视图内部坐标转换到显示坐标。

`View` 还有一些实例变量：`insetDisplayBox` 是视图可以绘制而不会覆盖其他视图的裁剪矩形；`borderWidth` 与 `borderColor` 定义视图和上层视图之间不参与缩放的边框；`insideColor` 定义实际绘制内容前用于填充 `insetDisplayBox` 的颜色（如果有）；`boundingBox` 描述视图坐标系中绘图对象所占的范围。

默认情况下，向视图发送 `model: anObject` 消息，会把该视图的控制器初始化为其默认控制器类的新实例；将 `anObject` 设为视图和控制器共同的模型；并把视图加入为模型的依赖对象。通常仅这一条消息，就足以建立 MVC 结构。

`release` 消息用于关闭视图层级（例如一个顶层窗口的全部子视图），并允许程序员执行必要的收尾工作。默认的 `release` 会断开视图与控制器之间的引用，也会把视图从模型的依赖对象集合中移除，从而消除它们之间的循环引用。

Controller 类（Class Controller）

控制器的工作，是处理某个模型及其特定视图的控制或操作（修改）功能。具体说，控制器配合模型、视图和输入设备，并处理调度任务。

抽象上层类 `Controller` 有三个实例变量：`model`、`view`、`sensor`。最后一个通常是 `InputSensor` 类的实例，表示输入设备的行为。

由于如何解释输入设备的操作高度依赖具体应用，`Controller` 本身几乎不实现这部分行为。它所实现的少量行为中，有一项是判断光标是否位于控制器对应的视图内。

Controller 类确实包含默认的调度行为。它假设任一时刻只有一个控制器处于活动状态，也就是只有一个控制器在解释用户输入。其他视图可以同时显示信息，但用户操作只由一个控制器解释。

因此，Controller 提供用于判断控制器是否需要取得或保持控制权的行为，还提供发出控制开始与结束信号的行为，因为许多控制器需要在开始或结束控制时执行特殊操作。

如果具体的控制器子类需要不同的控制权判定方式，可以覆盖 isControlWanted 和 isControlActive 这两个查找消息。默认情况下，它们使用 controlToNextLevel（继续向下转交）与 viewHasCursor（查找视图是否包含光标），使视图层级中包含光标的最底层子视图的控制器取得并保持控制权。

控制器一旦取得控制权，默认行为便是依次向它发送 controlInitialize、controlLoop 和 controlTerminate 消息。这一消息序列写在 startUp 方法中。

需要特殊启动和结束行为的子类会覆盖 controlInitialize 与 controlTerminate。例如，许多列表控制器只在取得控制权时才在左侧显示滚动条。

controlLoop 被实现为一个循环：只要控制器仍拥有控制权，就不断发送 controlActivity 消息。许多控制器类都会覆盖它，以完成用户界面真正的工作。

StandardSystemView 与 StandardSystemController

View 与 Controller 的各个子类实现更具体的视图和控制器行为。StandardSystemView 和 StandardSystemController 负责实现“窗口系统”行为。

StandardSystemController 定义了标准的 blueButtonMenu（通常关联Mouse右键），用于访问关闭、折叠和调整顶层视图大小的操作。它也负责把视图布置为活动视图之一。

StandardSystemView 负责顶层视图的标题、折叠或展开时的显示（可能使用图标），以及显示尺寸的最小值、最大值和变化方式。

交互式应用通常位于一个系统视图之内：应用的视图及控制器会作为应用顶层 StandardSystemView 的子视图而创建。

除 StandardSystemController 之外，系统还提供带默认行为的其他抽象控制器类。NoController 的实例永远不会取得控制权，经常与只读视图一起使用。

MouseMenuController 包含Mouse按键按下时显示浮动选项表的行为。由于用户界面大量使用浮动选项表，包括 StandardSystemController 在内的大部分控制器都是 MouseMenuController 的子类。另一个例子是实现滚动条的 ScrollController。

因为视图经常由多个部分组成（复合视图），一个应用视图可以包含多个子视图——例如图 2 的 FinancialHistoryView 包含两个条形图子视图——所以，在应用的 StandardSystemView 中，视图实例变量的含义可能与在子视图中的含义不同。

View 的众多子类中，有些用于充当顶层视图，例如 StandardSystemView；另一些则设计为结构化视图内部的单个子视图，可以作为组件插接到其他视图中，例如许多应用使用的 SelectionInListView。

图 4 展示了运行 CounterView 时有关 MVC 对象的状态及相互关系。通过实例变量与依赖对象关系，把模型、视图和控制器连接起来的箭头展示了这些对象之间的依赖关系。完整源代码列在本文后面的“MVC 实现示例”章节。

用户界面组件层级（User Interface Component Hierarchy）

View、Controller 及其他抽象类，共同形成 Smalltalk-80 MVC 实现中的视图和控制器框架。系统还提供不同的 View 和 Controller 子类，构成交互式应用所需的用户界面功能，例如选项表、文本和图形。

这些组件被组合起来，形成 Smalltalk-80 应用开发环境的标准系统视图；它们也可以由系统应用复用或进一步继承。它们构成了模型构建工具与用户交互组件库的核心。程序员能够像从架子上取工具一样，取出这些组件并将其插接起来，创建交互式应用。

下面列举其中一些组件。多数情况下我们只写 View 子类名称，默认它们与一个或多个合适的 Controller 子类配合使用。

SwitchView 和 ListView 是 View 的两个子类，分别提供静态或动态选项表行为。

SwitchView 例如用于系统浏览器的实例侧／类侧切换开关，或表单Editor的图标选项表。它像开关一样有开和关两种状态。当用户点击它时，它会通知自己的模型。这类视图通常用于文本选项表、图形选项表或“按键”。

ListView 是一个可滚动的文本项选项表，例如系统浏览器顶部的四个子视图（图 11）。当用户在列表中重新选择某项时，它会通知自己的模型。图 5 展示了 SwitchView 子类（绘图程序 FormEditor 的图标选项表）和 ListView 子类（Browser 的分类列表子视图）的使用。

Prompter 和 Confirmer 是用于取得用户输入的简单视图。启动它们时传入一个查找或提示字符组。前者能够返回字符组，后者返回布尔值。

浮动选项表更接近控制器与传感器之间的接口，因此被实现为一种特殊的 MVC 类。PopUpMenu 是 Object 的子类，具有自己的调度和绘制行为。用户按下Mouse键时，通常由 MouseMenuController 调用浮动选项表。

默认情况下，PopUpMenu 被调用后返回一个数字，调用它的控制器据此决定执行哪个操作。PopUpMenu 有多个子类，有的实现多级选项表，有的选择选项表项后返回符号而非数字。图 7 展示了典型选项表。

还有许多视图与控制器子类提供文本处理组件。StringHolderView 和 TextView 提供文本视图，用于 Workspace、Editor、Transcript 等展示文本的工具。控制器子类 ParagraphEditor 和 TextEditor 提供标准文本修改功能，供许多文本修改子视图复用。图 8 显示了基于文件的文本Editor视图布局及其组件名

称。

除上述视图类型外，系统还提供构建 MVC 界面所需的其他类，包括前面提到的特殊控制器类，以及用于监控用户输入的 `InputSensor`。

一个名为 `Sensor` 的全局对象是 `InputSensor` 类的唯一实例，用来表示用户的输入设备，例如Mouse和键盘。可以向它发送 `anyButtonPressed` 等查找消息，也可以通过 `waitBlueButton` 等消息等待用户操作。

通常不必直接引用 `Sensor`，因为基本控制器类已经使用了它。不过，许多专门的用户界面组件，尤其是直接追踪Mouse的组件（例如绘制动式直线），仍会使用它。

程序开发支持工具示例（Program Development Support Examples）

Smalltalk-80 的软件开发支持环境提供 Workspace、Inspector、Browser、Debugger，以及多种 Editor。它们展示了如何选择合适的模型，再从视图与控制器组件库中取出子视图加以组合，构建完整的工具。这里既有通用的对象检查工具，也有支持于事件列表等特定领域对象的工具。

工作区（Workspaces）

系统里的 Workspace 由 `StringHolderView` 与 `StringHolderController` 组合而成，作为 `StandardSystemView` 的唯一子视图布装。它们的模型是 `StringHolder` 类的实例，后者只保存一个 `Text` 实例，也就是附带格式信息的字符组。`StringHolderController` 实现的选项表消息，对应基本的文本修改命令与 Smalltalk-80 代码求值命令，如图 9 所示。

检查器（Inspectors）

系统中的 Inspector 由两个视图实现：左侧 `ListView` 显示实例变量列表；右侧 `TextView` 显示当前选中的实例变量的值。`InspectorView` 实例充当两者共同的上层视图，而 `StandardSystemView` 作为它的上层视图，负责窗口调度。这些视图的模型是一个 `Inspector` 实例。

Inspector 可以查看任何对象。独立的 `Inspector` 类充当中介或过滤器，负责访问任意对象的任意方面。这样便不必向 `Object` 类添加额外的消息共议。在视图与“真正的”模型之间使用中介对象，是进一步隔离显示行为与应用建模行为的一种常见做法。

通过针对特定的组合对象继承 Inspector 子类，可以为这些对象构建更合适的交互界面。例如，可为实例变量、数组元素和字典内容提供不同的检查方式。图 10 展示了普通检查器、数组检查器与字典检查器。

浏览器（Browsers）

与检查器类似，系统代码浏览器也采用中介对象来建立模型。每个系统 Browser 都有一个 `Browser` 类的实例作为中介模型，表示通过系统类层级查找类分类、类及其消息的路径。

Browser 模型的依赖对象包括：四个列表视图，分别显示类分类、类、消息共议和消息；一个代码（文本）视图，用于显示所选消息的方法代码或类定义模板；两个切换视图，用于选择浏览实例消息还是类消息（见图 11）。Browser 类为这些子视图分别提供不同的消息共议。

每个子视图通过向模型发送不同消息，查找系统代码的某一方面；每个控制器则在用户希望修改该方面时，发送相应的消息。

例如，当用户在类列表子视图中选择一个新类时，该子视图的控制器向 Browser 发送 `className: newClassName`。Browser 相应地设置自己的新状态，其中包括发送 `self changed: #protocol` 和 `self changed: #text` 消息。

收到对应的 `update:` 通知后，分类子视图、类/实例切换视图和类子视图不做任何事情。共议子视图向 Browser 请求新的共议列表并显示它。消息列表子视图清空内容。文本子视图向 Browser 请求当前文本，也就是新的类模板，并将其显示出来。这样，共用同一个模型的多组视图—控制器就共同产生了预期效果。

Smalltalk-80 系统还有其他类型的 Browser。留意前述系统 Browser 右上角消息列表中的中间一组选项表项：senders、implementors、messages。它们可以打开新的 MessageSetBrowser，只浏览系统消息的一个子集：当前所选消息的全部发送者、同一消息的其他实现者，或当前选中方法内部发送的全部消息。

另有一些 Browser 用于变更管理（ChangeSetBrowser）或系统恢复（ChangeListBrowser）。

调试器（Debuggers）

Debugger 类是 Browser 的子类，因此继承了交叉引用行为，例如通过选项表查找某个消息的发送者、实现者，以及某个方法使用的消息。它也继承并扩展了代码视图的行为。

顶部子视图是一个上下文列表，也就是消息发送序列。它使用 Debugger 类定义的消息共议，把系统的消息发送上下文调用序列显示成列表。

与系统 Browser 不同，调试器应用不只有 Debugger 这一个模型。它的两个 Inspector 部分各有独立的 Inspector 模型，而这两个部分又包含底部的四个子视图。

Debugger 实例把这两个 Inspector 对象保存为实例变量。当用户选择新的上下文时，Debugger 会显式改变这两个 Inspector 所检查的对象。这是多个模型对象共同工作、而视图各自独立但相互配合的例子。

它也体现了让 Inspector 类充当对象过滤器的好处：Inspector 对象正在检查的“真正模型”可以改变，但视图和控制器仍然引用同一个稳定的 Inspector 模型。

Smalltalk-80 应用中的对象Editor（Object Editors in Smalltalk-80 Applications）

除了浏览和检查，修改也是应用及软件开发工具最重要的方面之一。Smalltalk-80 标准Editor包括文本和文件Editor、用于图形的 Form 与位图Editor，以及用于源代码和资源管理的文件系统Editor。

很多基于 Smalltalk-80 的应用还为其领域特有的结构化对象开发新的图形Editor，例如图表、关系图、地图、网状结构、电子表格、动画、事件列表及数据库内容。

视图与控制器的拆分，以及可插接视图（View/Controller Factoring and Pluggable Views）

最初，编程环境工具采用这样的实现方式：模型完全不了解自己的视图和控制器；不同的行为主要通过继承不同子类来实现。

例如，在这种方式下，如何创建合适的浮动选项表，全部由控制器类决定。每一种不同的视图都有对应的专门类。每个控制器类都有类变量，用于保存选项表及选项表项所对应的消息；创建实例时，再把这些类变量关联到实例变量。每种不同的控制器类，还对应一个新的视图类。

部分系统视图至今仍采用这一实现方式。它有一些优势，例如系统组件的职责划分比较清晰。

然而，我们注意到，许多控制器类和视图类具有大量共同属性，特别是实现列表视图的那些类。其中一个共同点是，模型几乎总是某种过滤器类，已经知道列表本身以及列表中选中的项。

视图类通常完全相同，唯一的差别是用于创建不同控制器的 `defaultControllerClass` 消息。控制器之间也非常相似，区别只在于选项表项集合，以及选中选项表项时向自己发送的消息。

最后，这些控制器收到的选项表消息几乎总会直接转发给模型。也就是说，当选项表项 `aMessage` 被选中时，控制器收到 `aMessage`，而它对应的方法几乎总只是执行 `model aMessage`。

如果上述差别（例如向模型发送什么消息以生成列表）不必通过创建不同的视图/控制器类来表示，而可以成为同一个类的参数（实例变量），应用开发者的工作就会容易得多。这正是所谓的可插接视图（Pluggable Views）。

开发者不再通过创建两个新类来构建一种新的视图（例如新型列表视图），而是创建已有视图类的实例，并指定它应当如何与模型通信、如何响应选择、如何建立选项表。

从某种意义上说，这是一种工程权衡：它不如定义全新类那样灵活，也可能让一部分控制器相关信息进入模型。不过，它减少了开发者完成一个应用所需做的不同种类的工作，也减少了所需定义的类数量。

一个例子是系统 Browser 的列表子视图。原来的实现为每个列表子视图分别定义 `ListController` 子类。每个子类各自定义选项表、选择选项表项时发送的消息，以及选中新的列表项时发送的消息。

现在的可插接实现使用同一个 SelectionInListController 类的四个实例。每个实例通过参数指定：选择变化时应向模型发送什么消息，以及按下适当的Mouse键时如何创建相应选项表。Browser 模型知道四个局部选项表，并接收对应消息。

图 13 展示了如何用初始化消息将可插接的 SelectionInListView 加入复合视图。这段代码来自 BrowserView 实际的视图初始化方法。它在 myAreaRectangle 描述的子视图区域内定义一个 SelectionInListView，并设置图中列出的消息和选项表。

SelectionInListView 的可插接性由类消息 on:aspect:change:list:menu:initialSelection: 提供。View 类定义的 addSubView:in:borderWidth: 消息，则负责组合复杂的视图/子视图布局。

这样的消息正是子视图可插接性的核心。系统用户界面类中有大量实例可供研究。其他几种可插接子视图类也提供类似的实例创建消息。

另一个可插接视图的例子，是许多系统视图中使用的文本视图。我们希望把一个文本修改子视图插入复合视图，并告诉它：获取新文本内容、设置模型文本、显示选项表所需的消息。

CodeView 类为此实现了 on:aspect:change:menu:initialSelection:，请注意它与上面的 SelectionInListView 初始化消息有多相似。图 14 的代码就是定义 FileEditor 视图的完整方法；其界面如图 8 所示。

其他一些视图同样支持可插接初始化消息，包括开关视图（传入文字标记、查找状态和响应点击的消息）以及确认器和提示器（传入提示内容，返回字符组或布尔值）。

模型与 MVC 的使用（Models and MVC Usage）

Object 类包含支持 Model 功能的行为，也就是说，任何对象都可以拥有依赖对象。实现这一点所使用的机制，是一个将模型对象映射到依赖对象集合的字典 DependentFields。Object 还实现了添加和移除依赖对象的消息共议。

例如，某个对象 aModel 收到 addDependent: someView 时，someView 就会被加入 DependentFields 中以 aModel 为键所找到的集合。

由于视图和控制器直接保存指向模型的指针，DependentFields 字典会形成一种循环引用，而多数存储管理器无法回收这样的关系。release 机制的一个作用，就是移除对象的依赖关系，从而打破这种循环。因此，除非处于 MVC 应用开发过程，否则通常不会构成问题。

使用 Model 实例形成的循环引用则属于更直接的对象引用环，多数存储管理器能够回收。因此，我们鼓励使用 Model 并从它继承子类。

更高级的 MVC 应用还有许多复杂方面，本文没有涉及，例如窗口和视口、灵活的滚动框架、文本排版与字体，以及包含不参与缩放子视图的复合视图。这些问题可以通过 Smalltalk-80 系统中的使用实例或系

统应用示例来研究。

感兴趣的读者也可参阅 ParcPlace Systems 的 ParcPlace Newsletter（原名 Smalltalk-80 Newsletter）往期文章，以及 OOPSTAD 的 HOOPLA Newsletter（地址见参考文献）。

MVC 实现示例（MVC Implementation Examples）

下面给出三个 MVC 实现示例：第一个是完整但非常简单的 Counter（计数器）视图应用；第二个是使用可插接组件的新型应用视图 Organizer；第三个是较复杂的 FinancialHistory 应用（前文及图 2 已介绍）的视图组件代码节选。

示例一：CounterView（计数器视图）

Counter 教学示例是 Smalltalk-80 Version VI 2.2 标准发行包的一部分，最初由Dortmund大学的 Michael Hanus 编写。

它实现了一个模型（`Counter` 实例），保存一个简单的数值；一个视图（`CounterView`），显示这个数值；一个控制器（`CounterController`），通过选项表让用户增加或减少计数器的值。原文随后给出了三个类的完整源代码。

首先，在系统的 `Demo-Counter` 类分类中，把 `Counter` 定义为 `Model` 的子类。它有一个保存数值的实例变量 `value`：

```
Model subclass: #Counter
    instanceVariableNames: 'value'
    classVariableNames: ''
    poolDictionaries: ''
    category: 'Demo-Counter'
```

接着，为 `Counter` 添加初始化新实例和读取、修改数值的方法。`initialize` 把初始值设为 0；`value` 返回当前数值；`value: aNumber` 设置当前数值，并执行 `self changed` 以通知依赖对象重新显示。`decrement` 和 `increment` 分别通过 `value:` 减少或增加 1。类侧 `new` 返回一个经过 `initialize` 初始化的实例。

然后定义控制器类 `CounterController`，继承 `MouseMenuController`。它从上层类继承全部实例变量。`initialize` 调用上层类初始化后，配置一个包含 `Increment` 与 `Decrement` 的浮动选项表，并将选项表项对应到 `#(increment decrement)` 消息。

控制器的 `decrement` 和 `increment` 方法分别执行 `self model decrement` 与 `self model increment`，把操作交给模型。它还覆盖 `isControlActive`，在蓝色Mouse键未按下时保持控制权。

接下来定义 `CounterView`，它是没有额外实例变量的 `View` 子类。为它添加显示模型当前值的方法（原文 Listing 1）。`displayView` 取得视图的 `insetDisplayBox`，在该区域左边、距顶部约三分之一高度的位置，将字符组 `value:` 与模型数值拼接成 `DisplayText` 后绘制出来。

当模型发送 `self changed` 后，视图会收到更新消息。`CounterView` 的 `update: aParameter` 直接调用 `self display`，重新绘制全部内容。它的 `defaultControllerClass` 返回 `CounterController`。

最后，类侧 `open` 方法创建一个新的 `CounterView`，给它关联新建的 `Counter` 模型，设置边框和白色背景，再创建一个标题为 `Counter` 的 `StandardSystemView` 作为顶层窗口，把计数器视图加入其中，并调用顶层控制器的 `open`。得到的界面和选项表见图 15。

计数器示例讨论（Discussion）

前面的代码是一个最简单的完整 MVC 实现。假设我们希望在视图中放置按键，而不是用选项表项增加和减少计数器数值。借助可插接按键视图，只要为 `CounterView` 编写一个新的 `open` 方法就能容易做到，如原文 Listing 2 所示。

在 `openWithGraphicalButtons` 方法中，顶层视图由三个子视图组成：显示计数器数值的主视图，以及两个分别表示加号和减号的 `SwitchView`。子视图的位置由相对矩形定义，图 16 左侧展示了这些矩形的划分。

主视图放在右侧 60% 的区域。左侧上半部分（宽 40%、高 50%）放增加按键，左侧下半部分放减少按键。两个按键的 `onAction:` 分别执行 `aCounterView model increment` 与 `aCounterView model decrement`。因此，点击按键就会更新模型数值，进而触发视图更新。

示例二：分层文本 Organizer（Hierarchical Text Organizer）

第二个例子是在两层分级文本上实现一个简单 Browser 视图。它有两个子视图：主题列表，以及当前选中主题的文本视图。模型是 Organizer，它用一个字典保存组织结构，字典的键是文本主题，值是主题内容。键显示在主题列表中，值显示在文本视图中。图 17 展示其布局和选项表。

这里使用 Organizer，一方面是为了说明可插接视图更复杂的用法，另一方面是为了展示 MVC 类的职责划分。

在这个例子中，单独的 `Organizer` 类实现模型的功能，同时提供视图所需的信息，并定义两个子视图使用的选项表。

`Organizer` 有两个实例变量：`organization`（组织字典）与 `currentCategory`（当前选中的主题/分类）。它继承自 `Model`：

```
Model subclass: #Organizer
    instanceVariableNames: 'organization currentCategory'
    classVariableNames: ''
    poolDictionaries: ''
    category: 'Interface-Outlines'
```

Organizer 的基本方法负责初始化以及按分类访问数据。`initialize` 创建空字典。`organizationList` 返回字典键的排序集合。`organization:` 设置当前分类并发送 `self changed: #text`，通知文本视图更新。

addCategory 会通过 FillInTheBlank 问问新分类名称，为它在字典中建立空的 Text，将其设为当前分类，并发送 self changed: #organization。removeCategory 先用 BinaryChoice 请求确认，然后移除当前分类并广播组织变化。renameCategory 问问新名称，再转移原字典项的文本。organizationMenu 根据当前是否选中分类，返回包含添加、重命名、移除等不同选项表项的 ActionMenu。

文本相关消息（原文 Listing 3）允许用户查找与设置当前分类的文本。text 返回当前分类的文本备份；没有选择分类时返回空的 Text。acceptText: aText 接受文本子视图修改后的文本，将其复制进字典并返回成功状态；没有选择分类时返回不成功。textMenu 提供重复、取消、复制、剪切、粘贴、执行代码、打印、检查、接受与取消等选项表项。

解析输入流的方法假定，条目之间以及条目键与值之间采用特殊字符组分隔符。借助 parseFrom:entrySeparatorString:keySeparatorString:，Organizer 可以解析许多常见文件格式，例如系统 Workspace、密码文件或表格。

Organizer 的类侧消息负责创建新实例和加载普通文件。new 创建并初始化 Organizer。load: 打开文件，并以空行与双空行形式的分隔符解析内容。openFile: 载入指定文件并打开相应视图；openOn:label: 为给定 Organizer 创建带文字标记的顶层系统视图。

它使用可插接的 SelectionInListView 显示主题列表，通过 aspect: #organization、change: #organization:、list: #organizationList、menu: #organizationMenu 和 initialSelection: #currentCategory 指定访问与修改行为。

另一个可插接 CodeView 显示文本，使用 aspect: #text、change: #acceptText: 与 menu: #textMenu 配置。文本视图使用一种在修改后接受修改的控制器。主题列表占顶层窗口上方约 30%，文本视图占下方约 70%。最后启动顶层控制器。

例如，Organizer 可用来创建浏览 Smalltalk-80 系统 Workspace 内容的工具，如图 17 所示。

示例三：FinancialHistory 视图的建立（FinancialHistory Example View Setup）

接下来，原文以精简代码（Listings 4 和 5）介绍 FinancialHistory、FinancialHistoryView 与 FinancialHistoryController。更完整的讨论见 [Goldberg and Robson 83] 与 ParcPlace Systems Smalltalk-80 VI 2.3 发行文件。图 2 展示了这个例子的界面布局和标准选项表。

这里收录的是与 MVC 初始化及交互有关的方法。

控制器类实现 FinancialHistoryView 使用的默认选项表，负责问问用户，并向模型发送消息来改变状态，例如支出或收到一笔金。

FinancialHistoryController 继承自 MouseMenuController。它在初始化时建立 spend 与 receive 选项表，并将选项表选择映射为同名消息。

原文展示了控制器的 `receive` 方法：先通过 `FillInTheBlank` 问问收入来源，再问问收到多少金额；如果输入为空则结束；否则把字符组读取为 `Number`，然后向模型发送 `receive: amount from: receiveFrom`。

文中只展示控制器的 `receive` 消息；`spend` 消息与它非常相似。

`FinancialHistoryView` 类主要负责建立界面：它把两张 `BarChartView` 插入顶层视图，并启动适当的控制器。它的 `defaultControllerClass` 返回 `FinancialHistoryController`。

具体的类侧 `open: aFHModel` 方法先创建顶层 `StandardSystemView`，设置模型、边框、灰背景与 `Financial History` 标题。再创建 `FinancialHistoryView` 子视图，关联同一收支模型，并把它加入顶层窗口。

随后创建支出条形图：`BarChartView new model: aFHModel expenditures`，设为白色背景、宽度为 2 的边框，并使用 `NoController`（即它自己不处理输入）。它被放在复合视图的左侧区域。收入条形图同样创建，只是模型换成 `aFHModel incomes`，并放在右侧区域。最后执行 `topview controller open`，启动应用。

这三个例子显示了在 Smalltalk-80 系统中运用 MVC 范式与方法所能实现的部分复杂功能。我们鼓励读者浏览 Smalltalk-80 系统的界面类，或者阅读其他参考资料，了解更多 MVC 编程实例。

总结（Summary）

模型—视图—控制器构想，是一种设计并实现交互式应用软件的方式。它利用模块化，一方面帮助开发者在概念上构建应用，另一方面允许将已经为某个应用开发的组件复用到新的应用中。

这一构想要求把三方面的行为分开：应用领域的实际模型、用于显示模型状态的视图、以及对模型和视图进行修改或控制的部分。

我们已经在 Smalltalk-80 系统中实现了这一构想，并用它建立系统的基本编程开发工具，同时开发了多种不同类型的应用。

致谢（Acknowledgements）

Smalltalk-80 编程系统和 MVC 用户界面范式，是Xerox Palo Alto研究中心（PARC）系统概念研究组的许多成员在多年间共同开发的。

Xerox PARC 和 ParcPlace Systems 的多位同事查看了本文，并作出了贡献。我们感谢他们的帮助。

参考文献（References）

1. Adele Goldberg. Smalltalk-80: The Interactive Programming Environment. Addison-Wesley, Menlo Park, 1983.

2. Adele Goldberg and David Robson. Smalltalk-80: The Language and Its Implementation. Addison-Wesley, Menlo Park, 1983.

3. ParcPlace Systems Newsletters, Nos. 1–14. ParcPlace Systems, Palo Alto.

4. HOOPLA, OOPSTAD（Object-Oriented Programming for Smalltalk Application Developers）通信。

延伸阅读（Further Reading）

- Stephen T. Pope. Smalltalk-80 Applications Bibliography. Smalltalk-80 Newsletter #11, ParcPlace Systems, September 1987.

- Ralph E. Johnson. Model-View-Controller. University of Illinois at Urbana–Champaign, November 1987.

- Sam S. Adams. “MetaMethods: The MVC Paradigm.” HOOPLA.

若要进一步研究 Smalltalk-80 系统中的 MVC 类，最好的资料就是系统本身。例如，使用 `MessageSetBrowser` 查看可插接视图初始化消息的所有发送者，往往很有启发。可以从 `CodeView` 的 `on:aspect:change:menu:initialSelection:` 入手，也可以查看 `SelectionInListView`、`SwitchView` 中对应的消息。

读者还可以浏览 `FillInTheBlank`、`BinaryChoice` 等简单交互界面类的引用，或者系统应用视图的 `open` 方法。若想研究更高级的交互方式，检查 `controlActivity` 消息的各种实现也很有帮助。

译者的阅读提示（非原文）

1. 这篇论文讨论的是后来 Smalltalk-80 中的 MVC 实现：`Model`、`View`、`Controller` 抽象类，以及 `Dependent` 变更通知机制。它不能直接代替 Reenskaug 1979 年的原始文稿。

2. 这里的 `changed` / `update` 是具体的 Smalltalk 消息共议；它实现了模型与视图之间的通知机制，但不是所有叫作 MVC 的系统都必须采用同一种共议。

3. 作者特别强调的 `Pluggability`，本质是把原来靠子类区分的差异转成实例参数和消息选择子。这和现代界面组件的配置式复用有明显的思想联系。

4. 源文中的 Smalltalk 代码保留在 PDF 后附的原文扫描页中，译文保留核心声明和消息名称，并逐段解释代码的职责。原版扫描的个别字符识别错误没有当作真实代码翻译。

原始英文扫描页（供核对图表与源码）

以下保留原 PDF 的完整页面，包括图表、程序列表、代码注释与历史页码。前面的中文译文引用的图号均可在这些原页中找到。

A Cookbook for Using the Model-View-Controller User Interface Paradigm in Smalltalk-80

Glenn E. Krasner
Stephen T. Pope

Introduction

The user interface of the Smalltalk-80™ programming environment (see references, [Goldberg 83]) was developed using a particular strategy of representing information, display, and control. This strategy was chosen to satisfy two goals: (1) to create the special set of system components needed to support a highly interactive software development process; and (2) to provide a general set of system components that make it possible for programmers to create portable interactive graphical applications easily.

In this article we assume that the reader has a basic knowledge of the Smalltalk-80 language and programming environment. Interested readers not familiar with these are referred to [Goldberg and Robson 83] and [Goldberg 83] for introductory and tutorial material.

MVC and the Issues of Reusability and Pluggability

When building interactive applications, as with other programs, modularity of components has enormous benefits. Isolating functional units from each other as much as possible makes it easier for the application designer to understand and modify each particular unit, without having to know everything about the other units. Our experiences with the Smalltalk-76 programming system showed that one particular form of modularity—a three-way separation of application components—has payoff beyond merely making the designer's life easier. This three-way division of an application entails separating (1) the parts that represent the model of the underlying application domain from, (2) the way the model is presented to the user and from, and (3) the way the user interacts with it.

Author's Address: ParcPlace Systems, 2400 Geng Road Palo Alto, CA 94303, glenn@ParcPlace.com.

Smalltalk-80 is a trademark of ParcPlace Systems.

Model-View-Controller (MVC) programming is the application of this three-way factoring, whereby objects of different classes take over the operations related to the application domain (the model), the display of the application's state (the view), and the user interaction with the model and the view (the controller). In earlier Smalltalk system user interfaces, the tools that were put into the interface tended to consist of arrangements of four basic viewing idioms: paragraphs of text, lists of text (menus), choice "buttons," and graphical forms (bit- or pixel-maps). These tools also tended to use three basic user interaction paradigms: browsing, inspecting, and editing. A goal of the current Smalltalk-80 system was to be able to define user interface components for handling these idioms and paradigms once, and share them among all the programming environment tools and user-written applications using the methodology of MVC programming.

We also envisioned that the MVC methodology would allow programmers to write an application model by first defining new classes that would embody the special application domain-specific information. They would then design a user interface to it by laying out a composite view (window) for it by "plugging in" instances taken from the pre-defined user interface classes. This "pluggability" was desirable not only for viewing idioms, but also for implementing the controlling (editing) paradigms. Although certainly related in an interactive application, there is an advantage to being able to separate the functionality between how the model is displayed, and the methods for interacting with it. The use of pop-up versus fixed menus, the meaning attached to keyboard and mouse/function key, and scheduling of multiple views should be choices that can be made independently of the model or its view(s). They are choices that may be left up to the end user where appropriate.

The Model-View-Controller Metaphor

To address the issues outlined above, the Model-View-Controller metaphor and its application structuring paradigm for thinking about (and implementing) interactive application com-

ponents was developed. *Models* are those components of the system application that actually do the work (simulation of the application domain). They are kept quite distinct from *views*, which display aspects of the models. *Controllers* are used to send messages to the model, and provide the interface between the model with its associated views and the interactive user interface devices (e.g., keyboard, mouse). Each view may be thought of as being closely associated with a controller, each having exactly one model, but a model may have many view/controller pairs.

Models

The model of an application is the domain-specific software simulation or implementation of the application's central structure. This can be as simple as an integer (as the model of a counter) or string (as the model of a text editor), or it can be a complex object that is an instance of a subclass of some Smalltalk-80 collection or other composite class. Several examples of models will be discussed in the following sections of this article.

Views

In this metaphor, views deal with everything graphical: they request data from their model, and display the data. They contain not only the components needed for displaying but can also contain subviews and be contained within superviews. The superview provides ability to perform graphical transformations, windowing, and clipping between the levels of this subview/superview hierarchy. Display messages are often passed from the top-level view (the standard system view of the application window) through to the subviews (the view objects used in the subviews of the top-level view).

Controllers contain the interface between their associated models and views and the input devices (e.g., keyboard, pointing device, time). Controllers also deal with scheduling interactions with other view-controller pairs: they track mouse movement between application views, and implement messages for mouse button activity and input from the input sensor. Although menus can be thought of as view-controller pairs, they are more typically considered input devices, and therefore are in the realm of controllers.

Broadcasting Change

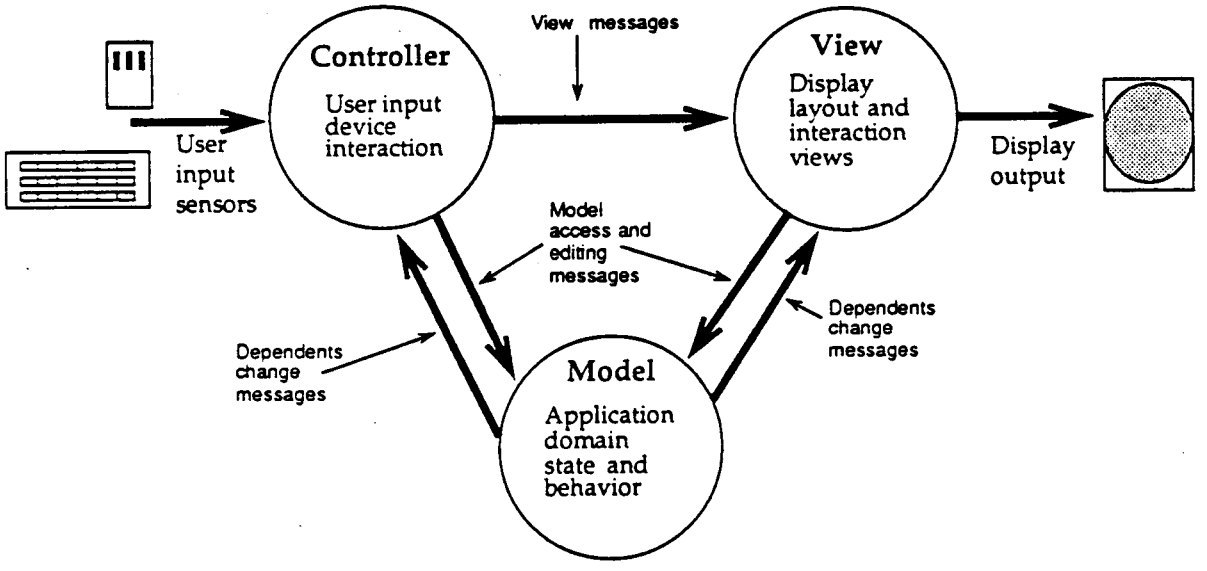
In the scheme described above, views and controllers have exactly one model, but a model can have one or several views and controllers associated with it. To maximize data encapsulation and thus code reusability, views and controllers need to know about their model explicitly, but models should not know about their views and controllers.

A change in a model is often triggered by a controller connecting a user action to a message sent to the model. This change should be reflected in all of its views, not just the view associated with the controller that initiated the change.

Dependents

To manage change notification, the notion of objects as *dependents* was developed. Views and controllers of a model are registered in a list as dependents of the model, to be informed whenever some aspect of the model is changed. When a model has changed, a message is broadcast to notify all of its dependents about the change. This message can be parameterized (with arguments), so that there can be many types of model

Figure 1. Model-View-Controller State and Message Sending



change messages. Each view or controller responds to the appropriate model changes in the appropriate manner.

A Standard for the Interaction Cycle

The standard interaction cycle in the Model-View-Controller metaphor, then, is that the user takes some input action and the active controller notifies the model to change itself accordingly. The model carries out the prescribed operations, possibly changing its state, and broadcasts to its dependents (views and controllers) that it has changed, possibly telling them the nature of the change. Views can then inquire of the model about its new state, and update their display if necessary. Controllers may change their method of interaction depending on the new state of the model. This message-sending is shown diagrammatically in Figure 1.

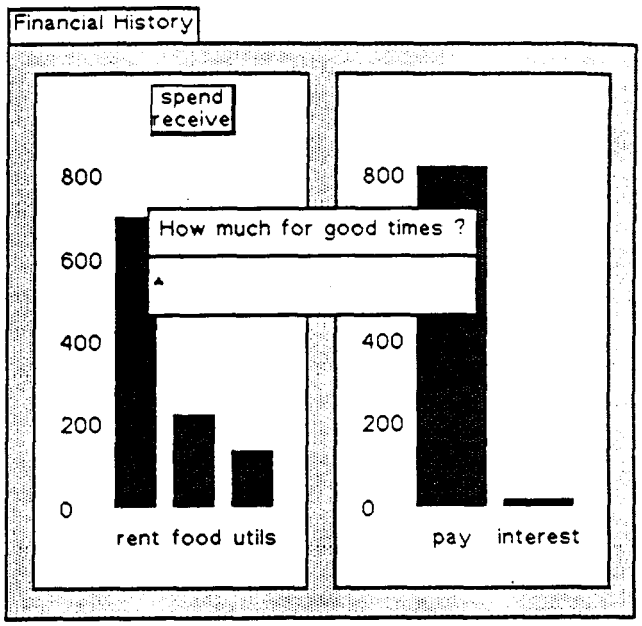
An Implementation of Model-View-Controller

The Smalltalk-80 implementation of the Model-View-Controller metaphor consists of three abstract superclasses named *Model*, *View*, and *Controller*, plus numerous concrete subclasses. The abstract classes hold the generic behavior and state of the three parts of MVC. The concrete classes hold the specific state and behavior of the application facilities and user interface components used in the Smalltalk-80 system. Since our primary set of user interface components were those needed for the system's software development tools, the most basic concrete subclasses of Model, View, and Controller are those that deal with scheduled views, text, lists of text, menus, and graphical forms and icons.

Class Model

The behavior required of models is the ability to have dependents and the ability to broadcast change messages to their

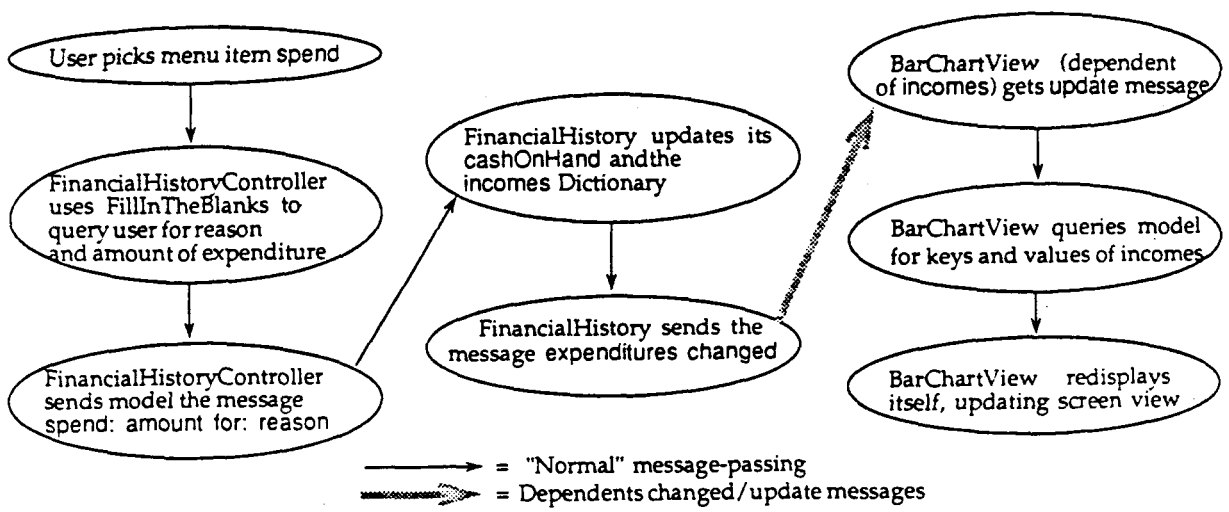
Figure 2. FinancialHistoryView with its BarChart subviews, the Controller's menu, and an interaction prompter (note that the menu and prompter are never visible at the same time)



dependents. Models hold onto a collection of their dependent objects. The class Model has message protocol to add and remove dependents from this collection. In addition, class Model contains the ability to broadcast change messages to dependents. Sending the message changed to a Model causes the message **update** to be sent to each of its dependents. Sending the message **changed: aParameter** will cause the corresponding message **update: aParameter** to be sent to each dependent.

A simple yet sophisticated MVC example is the FinancialHistoryview tutorial found in [Goldberg and Robson 83]. A display of a FinancialHistory is shown in Figure 2 and its implementation is discussed in the MVC implementation examples at the end of this article. In it, a view that displays a

Figure 3. Message-sending and dependency updating for an example from the FinancialHistory application



bar chart is created as a dependent of a dictionary of tagged numerical values (for example, **rent** ≥ $500). The composite FinancialHistoryView has two subviews, with two bar charts, whose models are two distinct dictionaries (incomes and expenditures). These are held as instance variables of the central model, an instance of class FinancialHistory.

Figure 2 also shows the pop-up menu used by the FinancialHistoryController with the two items labeled 'spend' and 'receive' as well as the input prompter (a FillInTheBlank) querying for an amount of be spent on 'good times'.

User action for interacting with the FinancialHistory application might be to pick an item from a menu to add a new amount for rent. The controller then sends a message to the model (the dictionary), and the model sends **self changed.** As a result of this, the bar chart is sent the message **update.** In response to that message, the bar chart gets the new values from the dictionary and displays the new bars using the display messages. A flow diagram for this MVC interaction might look like Figure 3.

The change messages with parameters (i.e., **self changed: someAspect**) are used to pass information from the model to its dependents about which aspect has changed, so as to minimize the amount of view updating needed. For example, the object householdFinances (the model mentioned earlier that holds onto the dictionaries of income and expenses), could have been the model of two bar chart views, with separate protocols for displaying expenses and income. In this case, the update message could contain a parameter saying which of the aspects (expenses or income) had changed.

Depending on the application, many of the basic Smalltalk-80 system structure classes can serve as models of MVC systems. Views can be found in the system or user applications that use very simple objects (numbers or strings), collection class instances (orderedCollections, dictionaries, bitmaps or displayTexts) or complex composite objects (networks, databases, event lists, or financial histories) as their underlying models.

Class View

The abstract superclass, class View, contains the generic behavior of views in the system. This includes model and controller interaction, subview and superview interaction, coordinate transformation, and display rectangle actions. The many subclasses of View implement the various display interaction tools used in the user interface.

Every instance of a view has exactly one model and exactly one controller. The model is normally set explicitly. Because view and controller classes are often designed in consort, a view's controller is often simply initialized to an instance of the corresponding controller class. To support this, the message **defaultControllerClass,** that returns the class of the appropriate controller, is defined in many of the subclasses of view.

Views have the ability to have zero or more subviews, with flexible coordinate transformations between one view and its super- and subviews. Instances of class View have instance variables for their superviews and for a (possibly empty) collection of subviews, as well as for an instance of class WindowingTransformation which represents the transformation (translation and scaling) between that view's coordinate system and that of its superview. In addition, there is a default protocol in View for adding and removing subviews, as well as a protocol for changing the transformations. This allows views consisting of many subviews to be pieced together flexibly and simply.

The third type of behavior in class View is that which relates to displaying. Because subclasses of View are assumed to use display objects to actually do their displaying (Forms, Pens, Lines, or instances of other graphical classes), View only supports generic displaying behavior. In particular, there is no instance variable for display objects. Class View assumes that the top level of a subview structure displays on some medium (typically the display screen).

Views therefore cache the transformation from their own internal coordinate system to the display's coordinate system (i.e., the composition of the view's transformation and all of its superviews' transformations), so that it is faster to get from a view's internal space to display space. View also has instance variables *insetDisplayBox*, the clipping rectangle within which the view may display without overlapping other views; *borderWidth* and *borderColor*, to define the (nonscaled) borders between a view and its superview; *insideColor*, to define the color (if any) with which a view colors its insetDisplayBox before actually displaying its contents; and *boundingBox*, to describe the extent of display objects within the view's coordinate system.

By default, the message **model: anObject,** when sent to a view, initializes the view's controller to be a new instance of the view's default controller class. It establishes *anObject* as the model for both the view and the controller, and establishes the view as a dependent of the model. This one message is typically sufficient for setting up the MVC structure.

The message **release,** used when closing a hierarchy of views (i.e., the subviews of one top-level window), gives the programmer the opportunity to insert any finalization activity. By default, **release** breaks the pointer links between the view and controller. The message **release** also removes the view from its model's dependents collection, breaking reference circularities between them.

Class Controller

It is a controller's job to handle the control or manipulation (editing) functions of a model and a particular view. In particular, controllers coordinate the models and views with the input devices and handle scheduling tasks. The abstract superclass Controller contains three instance variables: *model*, *view*, and *sensor*, the last of which is usually an instance of class InputSensor representing the behavior of the input devices.

Because the interpretation of input device behavior is very dependent on the particular application, class Controller implements almost none of this behavior. The one such behavior that is implemented is the determination of whether or not the controller's view contains the cursor.

Class Controller does include default scheduling behavior. It takes the point of view that only one controller is to be active at a time; that is, only one controller at a time mediates user

input actions. Other views could be displaying information in parallel, but the user's actions are to be interpreted by a single controller. Thus, there is behavior in class Controller for determining whether a controller needs to receive or maintain control. In addition, there is behavior to signal initiation and termination of control, since many controllers will need to have special initiation and termination behavior.

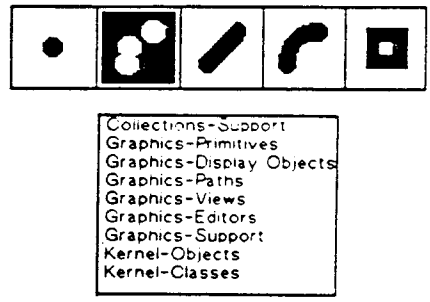
The query messages **isControlWanted** and **isControlActive** are overridden in concrete subclasses of Controller if they require a different behavior for determining whether to receive or maintain control. By default, these messages use the messages **controlToNextLevel** (pass it on down) and **viewHasCursor** (a query) in such a way that the controller of the lowest subview in the hierarchy that contains the cursor in its display box will take and retain control.

Once a controller obtains control, the default behavior is to send it the messages **controlInitialize, controlLoop,** and **controlTerminate.** This message sequence is found in the method **startUp.** The messages **controlInitialize** and **controlTerminate** are overridden in subclasses that want specific behavior for starting and ending their control sequences (for example, many of the list controllers have a scroll bar appear to their left only when they have control). **ControlLoop** is implemented as a loop that sends the message **controlActivity** as long as the controller retains control. This message is overridden in many controller classes to do the "real" work of the user interface.

StandardSystemView and StandardSystemController

Subclasses of View and Controller are the various classes that implement more specific types of view and controller behavior. Classes *StandardSystemView* and *StandardSystemController* are the implementation of "window System" behavior. StandardSystemController contains the definition of the standard *blue-button menu* (normally assigned to the right-hand mouse button), used for accessing operations of closing, collapsing, and resizing the top-level view on the display. It also contains the behavior for scheduling the view as one of the active views. StandardSystemView contains the behavior for top-level view labels, for displaying when collapsed or not (possibly using icons), and

Figure 5. SwitchView and ListView examples



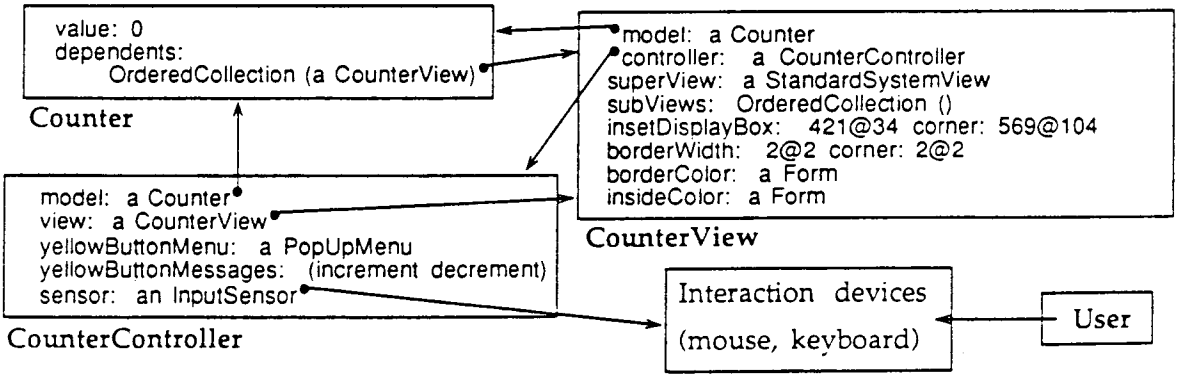
for the size (minimum, maximum, changing) of the view on the display. Interactive applications typically exist inside system views, that is, the views and controllers for the application are created as subviews of a StandardSystemView (the application's top-level view).

In addition to StandardSystemController, the system provides other abstract controller classes with default behavior. Instances of *NoController* will never take control; they are often used in conjunction with "read-only" views. Class *MouseMenuController* includes behavior for producing pop-up menus when any of the mouse buttons is pressed. Most controllers in the user interface are subclasses of MouseMenuController, including StandardSystemController itself, because of the extensive use of pop-up menus. Another example of these abstract classes is class *ScrollController*, which implements scroll bars.

Because views are often composed of parts (composite views), with one application view containing multiple subviews (as the FinancialHistoryView shown in Figure 2 has subviews for the two bar chart views), the instance variables for views have different meanings in an application's StandardSystemView than in their subviews. Of the many subclasses of View, some are meant to behave as top-level views (such as StandardSystemView), and others are meant to be used as subviews (single subviews within a structured view) and "plugged in" to other views as components (such as the SelectionInListViews used in many types of applications).

Figure 4 is a schematic diagram of the state and interrelationships of the relevant MVC objects used in the Counter-

Figure 4. Instance variables of an MVC Triad for running CounterView



The complete source code for this example is included below in the section called "MVC Implementation Examples." The interdependencies are shown by the arrows linking the model, view and controller together via instance variables and dependents.

User Interface Component Hierarchy

The classes View and Controller, along with the other abstract classes, provide the framework for views and controllers in the Smalltalk-80 implementation of the Model-View-Controller metaphor. The system also contains various subclasses of View and Controller that provide the pieces of user interface functionality required by interactive applications. These user interface components include menus, text, and forms. These components are pieced together to form the standard system views of the Smalltalk-80 application development environment, and can be reused or subclassed for system applications. They form the kernel of the component library of model-building tools and user interaction components that can be easily "pulled off the shelf" and "plugged" together to create interactive applications. We list some of them here, but in most cases we only use the names of the subclasses of View; it is assumed that each is used with one or more appropriate Controller subclasses.

SwitchView and *ListView* are two subclasses of View that provide static or dynamic menu behavior. SwitchViews are used, for example, in the instance/class switch of the system browser or the iconic menu of the form editor. They behave as a switch; they can either be on or off, and when the user clicks the mouse button on them, they will notify their model. These are typically used as menus of text or forms, or "buttons."

A ListView is a scrolling menu of text items, such as those used in the upper four subviews of the system browser (shown in Figure 11). It will inform its model when the user has made a new selection in the list. Figure 5 shows examples of the use of subclasses of SwitchViews (in the iconic menu of the FormEditor, the paint program) and ListViews (in the Browser's category list subview).

Prompters and *Confirmers* are simple views that can be used for eliciting user input. They are started up by giving them a string for their query or prompt, and can return either a string value (in the case of Prompters) or a Boolean value (in the case of Confirmers).

Because they can be considered as belonging more to the controller-sensor interface, pop-up menus are implemented as a special kind of MVC class. Class *PopUpMenu* is a subclass of **Object** and provides its own scheduling and displaying behavior. They are typically invoked by a MouseMenuController when the user pushes a mouse button. When invoked, PopUpMenus by default return a numerical value that the invoking controller uses to determine the action to perform. There are several subclasses of PopUpMenu, some that implement hierarchical (multi-level) menus and others that return symbols instead of numerical values upon item selection. Examples of typical menus are shown in Figure 7.

A number of view/controller subclasses provide text handling components. Class *StringHolderView* and class *TextView*

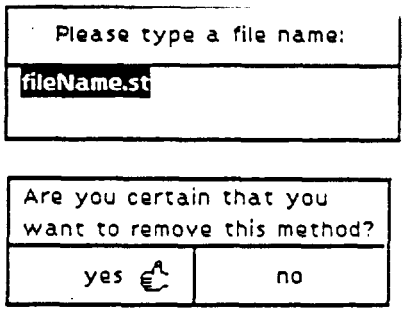


Figure 7. Examples of simple and hierarchical menus

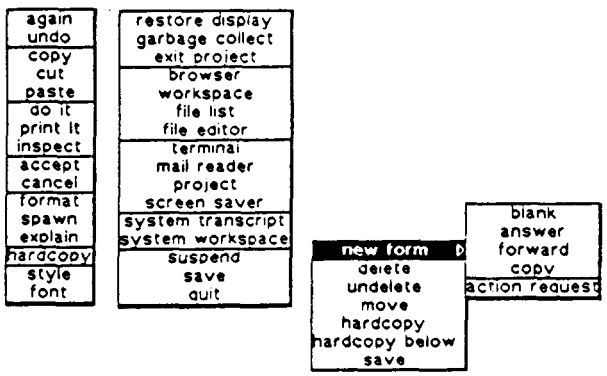


Figure 8. File-based TextEditorView and Menus

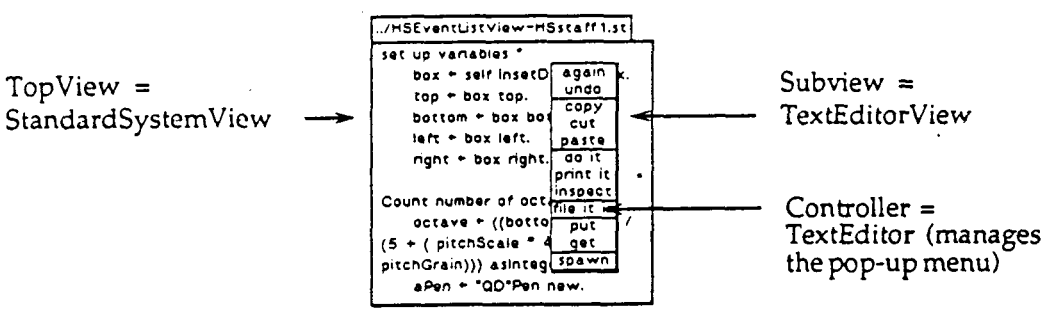
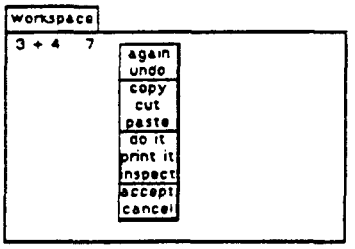


Figure 9. Simple Workspace View and Menu



provide views of text used in Workspaces, Editors, Transcripts, and other instances where text is displayed. The Controller subclasses *ParagraphEditor* and *TextEditor* have the standard text-editing functionality and are used in many text editor subviews. The layout of a file-based text editor view is shown in Figure 8 along with the names of its components.

In addition to these view types, there are several other classes provided within the system that are used in building MVC user interfaces. These include the special controller classes mentioned earlier: *MouseMenuController* and *NoController*; and the class *InputSensor* that is used for monitoring user input. There is a global object called *Sensor* that is the sole instance of the class InputSensor. It is used to model the user's input devices such as the mouse and the keyboard. One can send query messages to Sensor such as **anyButtonPressed**, or one can wait for a user action with messages such as **waitBlueButton**. One need normally never refer to Sensor, since it is used in the implementation of the more basic controller classes, but many types of special user interface components, especially those that track the mouse directly (for rubber-band lines, for example), use it.

Program Development Support Examples

Workspaces, inspectors, browsers, debuggers, and various editors are among the system views provided in the Smalltalk-80 software development support environment. They serve now as examples of piecing together subviews from the view and controller components collection with appropriate models to

Figure 10. Inspector Examples—Simple, Array and Dictionary Inspectors

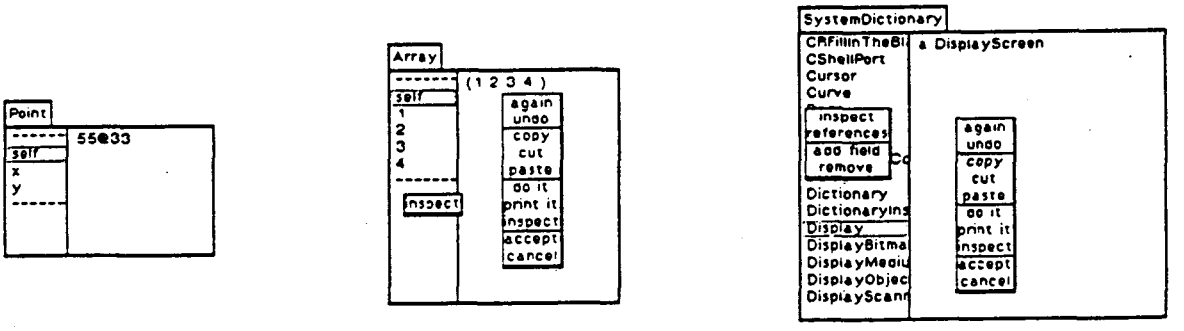
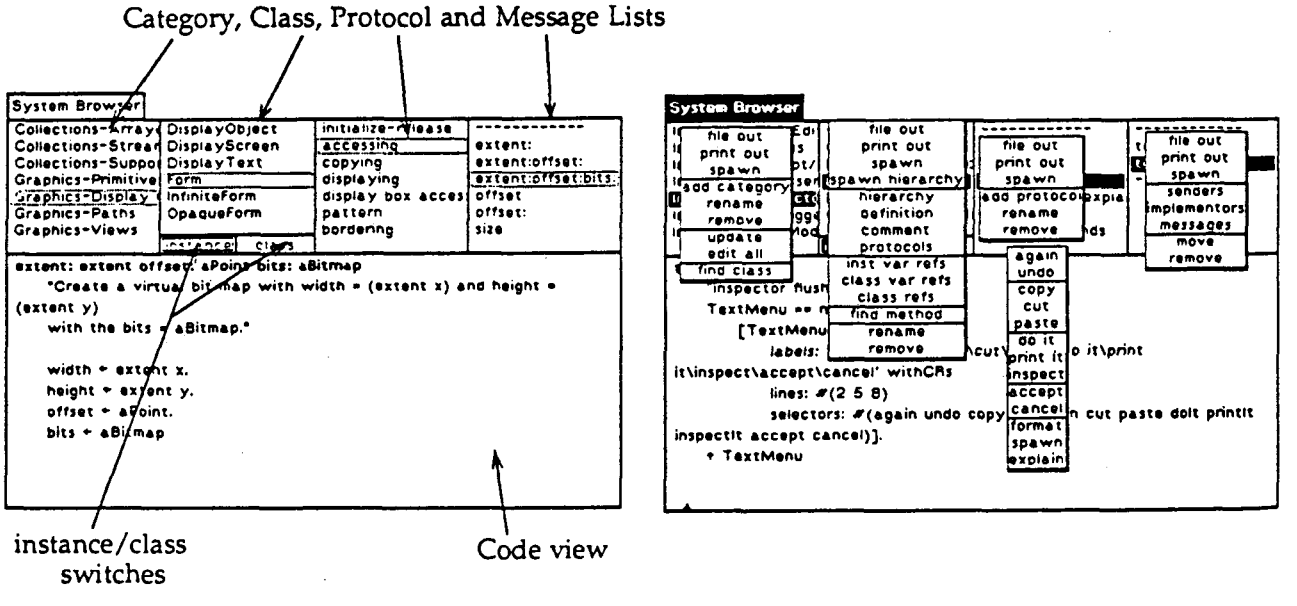


Figure 11. System Browser view layout and browser menus



tions.

Workspaces

The workspaces in the system are StringHolderView/StringHolder controller combinations installed as the single subview of a StandardSystemView. Their model is an instance of StringHolder, which merely holds onto an instance of Text, a String with formatting information. The menu messages implemented by StringHolderController correspond to the basic text editing and Smalltalk-80 code evaluation commands as shown in the menu in Figure 9.

Inspectors

The inspectors in the system are implemented as two views. A ListView contains the instance variable list (left side), and a TextView displays the value of the selected instance variable (right side). An instance of InspectorView serves as their common superview, and an instance of StandardSystemView serves as its superview for scheduling purposes. The model for these views is an instance of Inspector.

Inspectors can be used to view any object. A separate class, *Inspector*, serves as the intermediary or filter for handling access to any aspects of any object. As a result, no extra protocol is needed in class Object. Using intermediary objects between views and "actual" models is a common way to further isolate the viewing behavior from the modeling application.

It is possible to build a more appropriate interactive interface to composite objects by subclassing the inspector for use with

ized inspectors for complex objects such as ... selves or application-specific classes such as event lists.

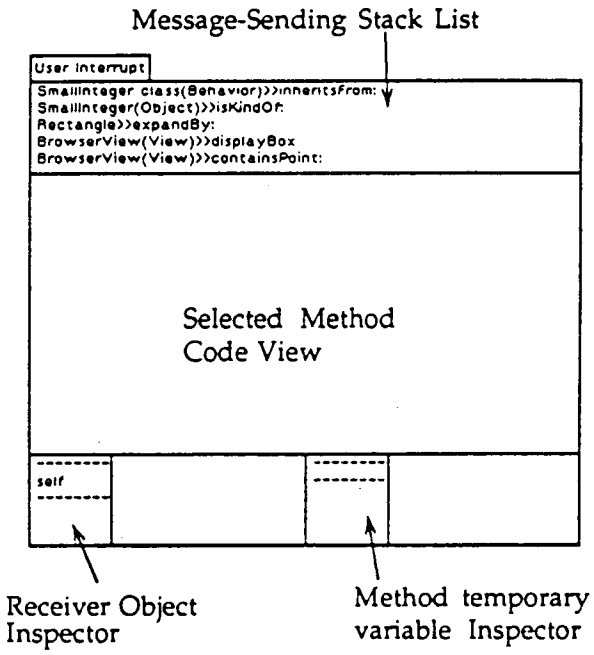
Browsers

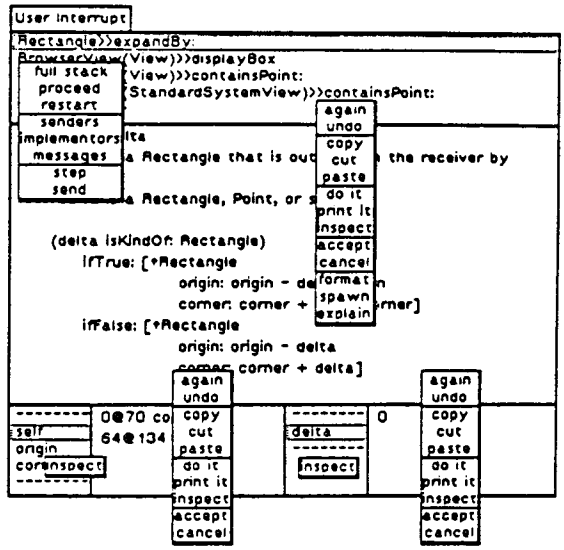
As with inspectors, intermediary objects are used to model the system code browser behavior. An instance of class *Browser* is the intermediary model for each system browser, representing the query paths through the system class hierarchy to the class organization and the classes. As dependents of the Browser model, there are the four list views (for Class Categories, Classes, Message Protocols and Messages), a code (text) view (for the method code of the selected message or a class definition template), and two switch views (for selective browsing of class and instance messages as shown in Figure 11). Class Browser has separate protocol for each of the various subviews of the browsers.

Each of the subviews sends different messages to the model to query the values of an aspect of the system code. Each of the controllers sends different messages when the aspect should be changed.

For example, when the user selects a new class in the class list subview, the controller for that subview sends the message **className: newClassName** to the Browser. The Browser sets up its new state accordingly. This involves sending the messages **self changed: #protocol** and **self changed: #text.** In response to the corresponding **update:** messages, the category subview, the class and instance switches, and the class subview do nothing. The protocol subview asks the Browser for the new list of protocols to display and displays it. The message list subview clears itself, and the text subview asks the Browser for

Figure 12. Debugger view layout and debugger's menus





the current text to display, which is the new class template. In this way, the six view/controller pairs sharing the single model work to produce the desired effect.

There are several other types of Browser found in the Smalltalk-80 system. Note the middle group of menu items in the upper-right subview (The MessageList) of the System Browser shown above. The menu items **senders, implementors,** and **messages** can be used to open new browsers (called *MessageSet Browsers)* on a small subset of the system messages—namely all the senders of the currently selected message, all other implementors of the same message, or all messages sent from within the method of the currently selected message. Other Browsers exist, for example, to assist in change management *(ChangeSet Browsers)* or system recovery *(ChangeList Browsers).*

Debuggers

Class *Debugger* is defined as a subclass of class Browser, so that it can inherit the cross-referencing behavior (menu messages for querying senders, implementors, and messages used in a method). It also inherits and augments the code view behavior. The upper subview, which is a context list (i.e., a message-sending stack), is a list view that uses protocol defined in class Debugger for displaying the system's message sending context stack as a list view.

Unlike the system browser, the Debugger is not the only model involved in the debugging application. There is a separate Inspector model for each of the two inspector subviews that comprise the four lower subviews. The Debugger instance holds onto those two Inspector objects as two of its instance variables: it explicitly changes the objects they are inspecting whenever a new context is selected. This is an example of using cooperating model objects with independent coordinated views. It also shows an advantage to having the Inspector class as a filter on the object: the object used as the "model" of the Inspector object can change, while the views and controllers refer to a consistent model.

Object Editors in Smalltalk-80 Applications

Along with the user interaction paradigms of browsing and inspecting, editing is one of the most important aspects of applications and software development tools. Among the standard editors available in Smalltalk-80 systems are text and file editors, form and bitmap editors for graphics, and file system editors for source code and resource management. Many Smalltalk-80-based applications implement new graphical editors for the structured objects that are specific to their application domains, such as charts, graphs, maps, networks, spreadsheets, animations, event lists, or database contents.

View/Controller Factoring and Pluggable Views

Originally, the program environment tools were implemented so as to have the models know nothing about their views and controllers and to use subclassing as the style for differentiating behavior. In this style, for example, all the knowledge for creating the appropriate pop-up menus is in the class, and there is a different class for each type of view. Each of these classes has class variables to hold the menu and the corresponding messages, and those class variables are bound to instance variables at instance creation time. Associated with each of these different controller classes was a new view class. This is still how some of the system views are implemented, and it has a number of advantages, such as clean factoring of system components.

We noticed, however, that many of these different controller and view classes shared a large number of properties, especially those that implemented list views. One similarity was that the models were almost always some sort of filter class that already knew about the lists and the selected item in the list.

The view classes tended to be identical except for the one message, **defaultControllerClass,** which is used to create the different controllers. The controllers were quite similar except for the particular set of menu items and the messages they sent themselves when an item was selected. Finally, the controller messages were almost always passed directly on to the model: that is, the method for message **aMessage,** which was sent to the controller when the menu item **aMessage** was selected, was almost always implemented as ↑model aMessage.

It would be easier for the application developer if these differences (e.g., the message sent to the model to generate the list) were not implemented by different view/controller classes, but were made parameters (instance variables) of a single class. This is the notion called *pluggable views.* Rather than building a new kind of view (e.g., a new kind of list view) by creating two new classes, the developer creates an instance of an exist-

Figure 13. Setup message for the class list view in the browser using a pluggable SelectionInListView

```
classListView ← SelectionInListView                "an instance of SelectionInListView"
        on: aBrowser                          "model of the SelectionInListView"
        aspect: #className                    "message to get the selected item"
        change: #className:                   "message sent on item selection"
        list: #classList                      "message sent to generate list"
        menu: #classMenu                      "message sent to get menu"
        initialSelection: #className.         "message sent to get initial selection"

self addSubView: classListView                     "Add a subview to the TopView"
        in: myAreaRectangle                   "relative area filled by SubView"
        borderWidth: 1                        "border to adjacent SubViews"
```

item selection definitions.

In some sense, this is an engineering trade-off, because it has less flexibility than entirely new class definitions and can lead to having controller information in the models. It does, however, reduce the number of different things the developer needs to do to get an application together, as well as the number of different classes needed.

An example of the use of pluggable views is the implementation of the system browser list subviews. The original implementation had a special subclass of ListController for each of the list subviews. Each of these classes had its own definition of the menus and messages to send when the menu item was selected, and its own message to send when a new list item was selected. The current pluggable implementation has four instances of the same class, SelectionInListController, with parameters that represent the messages to be sent to the model when the selection changes, and to create an appropriate menu when the proper mouse button is pressed. The Browser model knows about the four local menus and receives the corresponding messages.

The use of the setup message for adding a pluggable SelectionInListView to a composite view is demonstrated in the Figure 13. This code segment comes from the actual view initialization for the BrowserView. It defines a SelectionInListView in the subview area described by the rectangle **myAreaRectangle.** It uses the messages and the menu referred to in the figure.

The pluggability of SelectionInListViews is afforded by the class message shown here, namely **on:aspect:change:list:menu:initialSelection:.** The message **addSubView:in:borderWidth:** is defined in class View for the composition of complex view/subview layouts. Messages of this type are the essence of subview pluggability and examples of their use and utility are available through out the system's user interface classes. Several other classes of pluggable subviews implement similar instantiation *(plugging)* messages.

Another example of a pluggable view is the text view used in many system views. In this case, one wants to plug a text editor subview into a view and tell it the messages needed for it to access its new text contents, to set its model's text, and to display its menu. The message that is implemented in the class CodeView for this is **on:aspect:change:menu:initialSelection:** (note the similarity between this and the message used above for a pluggable SelectionInListView). The example message in Figure 14 is the entire method used to define a FileEditor view such as the one shown in Figure 8.

Several of the other views can be used with pluggable instantiation messages. These include the switch views (which one passes a label, the messages they send to determine their state and respond to being pressed), and confirmers and prompters (one passes them a message or prompt and they return strings or Boolean values).

Models and MVC Usage

Class Object contains behavior that supports Model's functionality, i.e., the ability for any object to have dependents, and able of class Object called *DependentFields*, which [illegible] models and whose corresponding values are collections of those models' dependents. Class Object also implements the message protocol to deal with adding and removing dependents. For example, when some object (like **aModel**) receives the message **addDependent: someView,** then **someView** is added to the collection found in the *DependentFields* dictionary at key **aModel.**

Since views and controllers hold direct pointers to their models, the DependentFields dictionary creates a type of circularity that most storage managers cannot reclaim. One byproduct of the release mechanism is to remove an object's dependents which will break these circularities, so this is typically not a problem except when developing an MVC application. The corresponding circularities that result from using instances of Model are the direct kind that most storage managers can reclaim. Therefore, we encourage the use and subclassing of Model.

There are several more sophisticated aspects of advanced MVC application that are not covered in this article. These include the use of windows and viewports, flexible scrolling frameworks, text composition and fonts, and view composition with nonscaling subviews. These issues can be studied via their usage within the Smalltalk-80 system or through examples found in Smalltalk-80 system applications. Interested readers are also referred to back issues of the *ParcPlace Newsletter (formerly the Smalltalk-80 Newsletter)* from ParcPlace Systems and the *OOPSTAD HOOPLA* Newsletter (see addresses in the reference list).

MVC Implementation Examples

Presented next are three MVC implementation examples: one a full application for a very simple view type (a Counter view); one a new application view using pluggable components (the Organizer view); and one a condensed listing for the viewing components of a more complex application (the FinancialHistory view discussed earlier and shown in Figure 2).

CounterView Example

The Counter demonstration and tutorial example is part of the standard Smalltalk-80 Version VI 2.2 release package and was originally written by Michael Hanus of the University of Dortmund. It implements a model (an instance of class *Counter)* that is a simple numerical value and view (a *CounterView)* on it which shows the value of the Counter. The controller *(CounterController)* implements a menu allowing one to increment or decrement the Counter's value. The complete code for these three classes follows.

First one must define a class named *Counter* as a subclass of Model in the system class category named *Demo-Counter.* Counter has one instance variable for its value.

```
Model subclass: #Counter
    instanceVariableNames: 'value'
    classVariableNames: ' '
```

Figure 14. Open Message for a FileEditorView using a Pluggable CodeView.

```
FileModel class methodsFor: 'instance creation'
open: aFileModel named: aString
"Scheduled a view whose model is aFileModel and whose label is aString."
| topView codeView |              "local variable for my top-level view and 1 subview"
                                  "set up the top-level standard system view"
topView ← StandardSystemView model: aFileModel
        label: aString
        minimumSize: 180@180.
codeView ← CodeView                "pluggable CodeView setup message"
        on: aFileModel             "it takes its model and the following:"
        aspect: #text              "message sent to the model to get the text"
        change: #acceptText:from:  "message sent to accept a new text"
        menu: #textMenu            "message sent to get text view's menu"
        initialSelection: nil.     "initially-selected text"
topView addSubView: codeView       "add the code view as the sole subview"
        in: (0@0 extent: 1@1)      "use the entire view's area"
        borderWidth: 1.            "with a 1-pixel border"

topView controller open            "open the default controller to start up view"
```

```
    poolDictionaries: ' '
    category: 'Demo-Counter'
```

Next, one adds methods to Counter for initializing new counter instances and for accessing their values. These messages will then be understood by all Counter instances.

```
Counter methodsFor: 'initialize-release'
initialize
    "Set the initial value to 0."
    self value: 0

Counter methodsFor: 'accessing'
value
    "Answer the current value of the receiver."
    ↑value

value: aNumber
    "Initialize the counter to value aNumber."
    value ← aNumber.
    self changed"to update displayed value"

Counter methodsFor: 'operations'
decrement
    "Subtract 1 from the value of the counter."
    self value: value -1

increment
    "Add 1 to the value of the counter."
    self value: value + 1
```

Add the method to class Counter to be used for getting a new counter instance.

```
Counter class methodsFor: 'instance creation'
new
    "Answer an initialized instance of the receiver."
    ↑super new initialize
```

Now define a class for the controller, along with the methods to define the menu it uses and those that implement the menu functions by passing them along to the model. The controller inherits all its instance variables from its superclasses.

```
MouseMenuController subclass: #CounterController
    instanceVariableNames: ' '
    classVariableNames: ' '
    poolDictionaries: ' '
    category: 'Demo-Counter'

CounterController methodsFor: 'initialize-release'
initialize
    "Initialize a menu of commands for changing the
    value of the model."
    super initialize.
    self yellowButtonMenu: (PopUpMenu labels:
        'Increment\Decrement' withCRs)
        yellowButtonMessages: #(increment decrement)

CounterController methodsFor: 'menu messages'
decrement
    "Subtract 1 from the value of the counter."
    self model decrement

increment
    "Add 1 to the value of the counter."
    self model increment

CounterController methodsFor: 'control defaults'
isControlActive
    "Take control when the blue button is not pressed."
```

```
↑super isControlActive &
sensor blueButtonPressed not
```

Next, define the class CounterView as a subclass of View with no additional instance variables.

```
View subclass #CounterView
  instanceVariableNames: ' '
  classVariableNames: ' '
  poolDictionaries: ' '
  category: 'Demo-Counter'
```

Add to it methods for displaying the state of its model (a Counter) in the view shown in Listing 1.

Define a method for updating the view when the model changes. The model's sending a **self changed** message will cause the view to be sent an update message.

```
CounterView methods for 'updating'
update: aParameter
  "simply redisplay everything."
  self display
```

Another method is needed to return the class of the default controller used within a counterView.

```
CounterView methodsFor: 'controller access'
defaultControllerClass
  "Answer the class of a typically useful controller."
  ↑CounterController
  CounterController'initialize-release'
```

Finally, a method is needed to open up a new counterView and set up the model and controller for it. The resulting view and its menu are shown in Figure 15.

Discussion

The code presented so far is the most trivial sort of complete MVC implementation. Suppose now that we wish to add push-buttons to the view instead of having menu items to increment and decrement the value of the counter. Using pluggable button views, this can easily be done by writing a new open method for the CounterView, as shown in Listing 2.

In the open method, one sees the setup of the view as a composition of its three subviews. The subview placement is done via the definition of relative rectangles. These relative rectangles are displayed in the left-hand figure in Figure 16. The definitions of the two SwitchViews and their Buttons sets their actions so that they send the **increment** and **decrement** messages to the model of the view. This will then have the desired effect of changing the value of the model (a Counter).

Hierarchical Text Organizer

The second example is the implementation of a simple browser view on a 2-level hierarchical text. It presents a view with two subviews: a list of topics and a text view for the selected topic's text. The model is an organizer, which holds onto its organization in a dictionary of text keys and text values. The keys are used in the topic list view and the values are the contents of the text view. The layout and menus of an organizer are shown in Figure 17.

The Organizer is included here as an example of a more sophisticated use of pluggable views and also as an example of MVC class factoring. In this example, the single class (Organizer) implements the functionality of the model and the view and also defined the menus used in the view's two subviews.

The organizer class has two instance variables; its organization dictionary and the currently selected category (topic, section).

```
Model subclass: #Organizer
  instanceVariableNames:'organization'
  currentCategory'
  classVariableNames:''
  poolDictionaries:''
  category: 'Interface-Outlines'
```

The most basic messages to an organizer are for setting it up and for accessing the organization by category.

```
Organizer methodsFor: 'initialize-release'
initialize
```

Listing 1.

```
CounterView MethodsFor: 'displaying'
displayView
  "Display the value of the model in the receiver's view."
  |box pos displayText|
  box ← self insetDisplayBox.          "get the view's rectangular area for displaying"

                                       "Position the text at the left side of the area, 1/3 of the way down"
  pos ← box origin + (4 @ (box extent y / 3)).
                                       "concatenate the components of the output string and display them"
  displayText ← ('value:, self model value printString) asDisplayText.

displayText displayAt: pos
```

Listing 2.

```
CounterView class methodsFor: 'instance creation'
openWithGraphicalButtons

"Open a view for a new counter that has fixed graphical buttons (whose forms are
  generated from the '+' and '–' characters and displayed on white backgrounds)
  for incrementing and decrementing the value."

"CounterView openWithGraphicalButtons"

  | aCounterView topView incrButton decrButton incrSwitchView decrSwitchView |
                                                    "top view = StandardSystemView"
  topView ← StandardSystemView new label: 'Counter'.

  topView minimumSize: 120 @ 80.
  topView maximumSize: 600 @ 300.
  topView borderWidth:2                             "set window border"

                                                    "main counter subview"
  aCounterView ← CounterView new model: Counter new.
  aCounterView insideColor: Form white.
                                                    "add main CounterView to topView in the right-hand 60%"
  topView addSubView: aCounterView
    in:(0.4 @ 0 extent: 0.6 @ 1)
                                    "a view's area is defined to be"
    borderWidth:0
                                    "the rectangle 0@0 to 1@1"

  incrButton ← Button newOff
                                    "define increment button and give it its action Buttons are used in Switches"
  incrButton onAction: [aCounterView model increment].
                                                    "put it in a switchView"
  incSwitchView ← SwitchView new model: incrButton
                                                    "whose label is a form"
  incrSwitchView label: ('+' as DisplayText magnifyBy: 2@2).
                                                    "surrounded by white"
  incSwitchView insideColor: Form white
                                                    "add the increment switch to topView"
  topView addSubView: incrSwitchView
    in: (0@0 extent: 0.4 @ 0.5)                     "put it in the top-left corner"
    borderWidth: (0@0 extent: 2@1).                 "Border is defined as left, top, right, bottom"
  decrButton ← Button newOff.                       "define the decrement switch"
  decrButton onAction: [aCounterView model decrement].

  decrSwitchView ← SwitchView new model: decrButton.

                                                    "its form is also put in there"
  decrSwitchView label: ('–' asDisplayText magnifyBy: 2@2).
  decrSwitchView insideColor: Form white
  topView addSubView: decrSwitchView                "add it in the lower-left"
    in: (0 @ 0.5 extent: 0.4 @ 0.5)                 "under the increment button"
    borderWidth: (0@1 extent: 2@0).                 "start up topView's controller"
  topView controller open
```

```
Organizer class methodsFor: 'creation'
new
    "make a default new Organizer"
    ↑super new initialize

Organizer class methodsFor: 'loading'
load: aFileName
    "Read a new Organization in from the given file using empty lines and double empty lines as the default
    separators. Many other formats can be parsed"
    "Organizer load: 'DT.ws'."
    |file org cr|
    file ← (FileStream oldFileNamed: aFileName).
    cr ← Character cr.
    org ← self new.
    org parseFrom: file
    entrySeparatorString: (String with: cr with: cr with: cr)
    keySeparatorString: (String with: cr with: cr).
    ↑org

Organizer class methodsFor: 'view creation'
openFile: aName
    "read a new Organizer from the given file"
    "Organizer openFile: 'DT.ws'."

    ↑self openOn: (self load: aName) label: aName

openOn: anOrganization label: aLabel
    "open an Organizer view on the given organization"
    "Organizer openOn: Organizer new label: 'Maintenance'"

    |topView listView textView|

    topView ← StandardSystemView          "top-level view"
        model: anOrganization
        label: aLabel
        minimumSize: 250@250.
    topView borderWidth: 1.

    listView ← SelectionInListView        "plug in topic list view"
        on: anOrganization                "model of list"
        aspect: #organization
        change: #organization:            "message sent to set new list"
        list: #organizationList           "message sent to get list"
        menu: #organizationMenu           "message sent to get menu"
        initialSelection: #currentCategory.
    textView ← CodeView                   "plug in text editor view"
        on: anOrganization                "with its model"
        aspect: #text                     "and its aspect accessing message"
        change: #acceptText:              "and change message"
        menu: #textMenu.                  "and its menu accesdsing message"
                                          "plug in a special controller for the text view"
    textView controller: AlwaysAcceptCodeController new.
                                          "plug the subviews into the top view"
    topView addSubView: listView
        in: (0@0 extent: 1@0.3)           "list view in the top 30%"
        borderWidth: 1.

    topView addSubView: textView
        topView controller open
        in: (0@0.3 extent: 1@0.7)         "text view in the bottom 70%"
        borderWidth: 1

topView controller open
```

Figure 15. View layout and menu of the simple CounterView

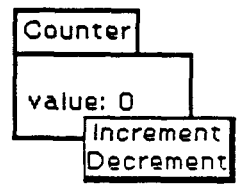
```
CounterView class methodsFor: 'instance creation'
open
        "Open a view for a new counter."
        "select and execute this comment to test this method"
                "CounterView open."

        | aCounterView topView |
                                        "create the counter display view"
aCounterView ← CounterView new                  "a new CounterView instance"
        model: Counter new.                     "with a Counter as its model"

aCounterView borderWidth: 2.                    "give it a borderWidth"
aCounterView insideColor: Form white.           "and white insides"

                                        "the top-level view"
topView ← StandardSystemView new                "a new system window"
        label: 'Counter'.                       "labelled 'Counter' "

topView minimumSize: 80@40.                     "at least this big"
                                        "add the counterView as a subView"
topView addSubView: aCounterView.
                                        "start up the controller"
topView controller open
```




```
    "set up a new empty Organizer. Its organization is
    an empty dictionary."
    organization ← Dictionary new

Organizer methodsFor: 'organization list'
currentCategory


organizationList
    "return the list of organization keys (topics), the
    keys of the dictionary"
    ↑organization keys asSortedCollection

organization: aCategory
    "set the current category and signal that the or-
    ganization text has changed"
    currentCategory ← aCategory.
    self changed: #text

addCategory
    "Add a new category, prompting the user (with a Fill-
    InTheBlank) for its name"
    |newCategory|
    NewCategory ← FillInTheBlank
    request: 'New Category' initialAnswer: (').
    newCategory =" ifTrue: [↑self].
    organization at: newCategory put: Text new.
    currentCategory ← newCategory.
    self changed: #organization

removeCategory
    "prompt the user and remove the current category
    from the organization"
    (BinaryChoice message: 'Are you certain that you
    want to remove category', currentCategory, '?')
    ifTrue:
      [organization removeKey: currentCategory.
      currentCategory ← nil.
      self changed: #organization]

renameCategory
    "prompt the user for a new name and rename the
    current category"
    |newCategory|
    newCategory ← FillInTheBlank
    request: 'New Category'
      initialAnswer: (currentCategory).
    newCategory = '' ifTrue: [↑self].
    organization at: newCategory put: (organization at:
    currentCategory).
    organization removeKey: currentCategory
    currentCategory ← newCategory.
    self changed: #organization

organizationMenu
    "return the menu to be used in the topic key list"
    currentCategory == nil
    ifTrue: [↑ActionMenu ActionMenulabels: 'add
    category' selectors: #(addCategory)].
    ↑ActionMenu ActionMenulabels: 'add
    category\rename\remove' withCRs
    selectors: #(addCategory renameCategory remove-
    Category)
```

Figure 16. Coordinate system and resulting view layout of a Counter View with graphical buttons

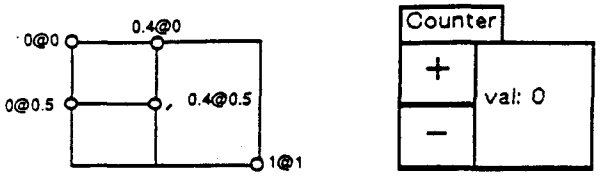
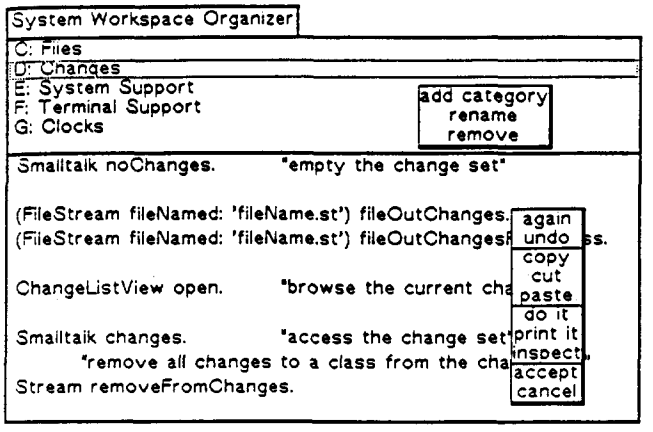


Figure 17. Organizer view showing list and text views and menus



The text-related messages allow the user to query and set the text value for the currently selected category, as shown in Listing 3.

```
Organizer methodsFor: 'text'
text
    "answer the text for the current category"
    currentCategory == nil ifTrue: [↑Text new].
    ↑organization at: currentCategory copy

acceptText: aText
    "this is sent to accept the changed text from the
    text subview"
    currentCategory == nil ifTrue: [↑false].
    organization at: currentCategory put: aText copy.
    ↑true

textMenu
    "answer the menu used in the text subview"
    ↑ActionMenu
        labels: 'again\undo\copy\cut\paste\do it\print it\in-
        spect\accept\cancel'
        withCRs
        lines: #(2 5 8)
        selectors: #(again undo copySelection cut paste
        doIt printIt
        inspectIt accept cancel)
```

The methods used to parse streams assume that special strings are used for separating entries from their keys and for separating many common file formats (such as System Workspaces, password files, or tables) to be parsed with organizers.

```
Organizer methodsFor: 'parsing'
parseFrom: aStream entrySeparatorString: entryStr
keySeparatorString: aKeyStr
    "read an organization from the given stream using
    the two given strings to
    parse the contents into entries and values"

    |tmp key body|
    [aStream atEnd] whileFalse:
        [tmp ← ReadStream on: (aStream upToAll: entry-
        Str).
        key ← tmp upToAll: aKeyStr.
        body ← tmp upToEnd asText.
        organization at: key put: body]
```

The class messages for organizers provide for the creation of new instances and the simple loading of standard files.

The organizer described above can be used, for example, for creating a browser on the contents of the Smalltalk-80 system's System Workspace, as shown in Figure 17.

FinancialHistory Example View Setup

On the following pages is a condensed version of the source code (Listings 4 and 5) for the classes FinancialHistory, FinancialHistoryView, and FinancialHistoryController as described in depth in [Goldberg and Robson 83] and the ParcPlace Systems Smalltalk-80 VI 2.3 release fileset. Figure 2 shows the view layout and standard menu for the FinancialHistory example. Included here is the method text for the MVC-related setup and interaction messages.

The controller class implements the default menus for use within FinancialHistoryView as shown below. It carries out user queries and sends messages to the model to change the state (such as after spending or receiving money).

Only the **receive** message for the controller is shown above; the **spend** message is closely analogous to it.

The class FinancialHistoryView simply contains the view setup message for plugging the two BarChartViews into a topView and starting the appropriate controller.

```
View subclass: #FinancialHistoryView
    instanceVariableNames: ' '
    classVariableNames: ' '
    poolDictionaries: ' '
    category: 'Demo-FinancialTools'

FinancialHistoryView methodsFor: 'controller access'
defaultControllerClass
    ↑FinancialHistoryController
```

The setup message defines the topView and inserts the subviews into it. The BarChartViews are defined in the support

Listing 4.

```
MouseMenuController subclass: #FinancialHistoryController
  instanceVariable Names: ' '
  classVariableNames: 'FHYellowButtonMenu FHYellowButtonMessages'
  poolDictionaries: ' '
  category: 'Demo-FinancialTools'

FinancialHistoryController methodsFor: 'initialize-release'
initialize
  "initialize me and set up the appropriate menus"
  super initialize.
  self initializeYellowButtonMenu

FinancialHistoryController methodsFor: 'private'
initializeYellowButtonMenu
  "plug in my menu and its messages from the class variables"
    "The message yellowButtonMenu: yellowButtonMessages: is implemented for all mouse-menu-controllers"
  self yellowButtonMenu: FHYellowButtonMenu
    yellowButtonMessages: FHYellowButtonMessages

FinancialHistoryController Class methodsFor: 'class initialization'
initialize
  "Specify the yellow button menu items and actions."
  FHYellowButtonMenu ← PopUpMenu labels: 'spend\receive' withCRs.
  FHYellowButtonMessages ← #(spend receive).

FinancialHistoryController methodsFor: 'menu messages'
receive
  "Ask what amount is being received from what and send the appropriate message to the model."

  | receiveFrom amount |
                                          "prompt the user with a FillInTheBlank prompter"
  receiveFrom ← FillInTheBlank request: 'Receive from what?'.
  receiveFrom =" ifTrue: [↑self].          "return if he/she answers blank"

  amount ← FillInTheBlank request: 'How much from', receiveFrom, '?'.
  amount =" ifTrue: [↑self].
                                          "read a number out of this string"
  amount ← Number readFrom: (ReadStream on: amount).

  model receive: amount from: receiveFrom.          "send it on to the model"
```

classes for the FinancialHistory example and are the bar chart elements seen in Figure 2.

The three examples presented here show some of the sophistication possible using the Model-View-Controller paradigm and methodology in the Smalltalk-80 system. Readers are encouraged to browse the Smalltalk-80 system interface classes or read the other references to see many more examples of MVC programming.

Summary

The Model-View-Controller metaphor is a way to design and implement interactive application software that takes advantage of modularity, both to help the conceptual development of the applications, and to allow pieces already developed for one application to be reused in a new application.

The metaphor imposes a separation of behavior between the actual model of the application domain, the views used for displaying the state of the model, and the editing or control of the model and views.

We have implemented the metaphor in the Smalltalk-80 system and have used this implementation both to create the basic programming development tools used in the system, and to develop a diverse collection of applications.

Acknowledgements

The Smalltalk-80 Programming System and the MVC user interface paradigm were developed over several years by numerous members of the Systems Concepts Laboratory at the Xerox Palo Alto Research Center (PARC).

Several of our colleaques at Xerox PARC and ParcPlace Systems reviewed and contributed to this paper. We are indebted to them for their assistance.

```
FinancialHistoryView class methodsFor: 'instance creation'
  open: aFHModel
  "open and schedule the MVC application for the Financial History given as the argument to this message"
  |aFHView aBCView topView|                    "define the top view (application window)"
  topView ← StandardSystemView new.
  topView model: aFHModel.
  topView borderWidth: 2.                          "plug in the model"
  topView insideColor: Form lightGray.             "make the background light gray"
  topView label: 'Financial History'.
                                                   "label the view"
  topView minimumSize: 400@300.                    "make it big"
                                                   "make the FHView for insertion into topView"
  aFHView ← FinancialHistoryView new model: aFHModel.
                                                   "add the FHView as a subview of topView"
  topView addSubView: aFHView.                     "use the entire area of topView"
                                                   define the expenditures BarChart"
  aBCView ← BarChartView new model: aFHModel expenditures.
                                                   "its model is the expenditures dictionary"
                                                   "its area is the given absolute region"
  aBCView insideColor: Form white.
  aBCView borderWidth: 2.
                                                   "it has no controller"
  aBCView controller: NoController new.
                                                   "add it as a subview"
  aFHView addSubView: aBCView
    in: (0.04@0.05 extent: 0.44@0.9)
    borderWidth: 2
                                                   "define the incomes BarChart similarly"
                                                   "its model is the incomes dictionary"
  aBCView ← BarChartView new model: aFHModel incomes.
  aBCView insideColor: Form white.
  aBCView borderWidth: 2.
  aBCView controller: NoController new
                                                   "add it as a subview"
  aFHView addSubView: aBCView
    in: (0.52@0.05 extent: 0.44@0.9)
    borderWidth: 2.
    "open the new top-level controller for the application:
  topView controller open
```

References

Adele Goldberg. *Smalltalk-80: The Interactive Programming Environment*. Addison-Wesley Publishers, Menlo Park, 1983.

Adele Goldberg and David Robson. *Smalltalk-80: The language and its Implementation*. Addison-Wesley Publishers, Menlo Park, 1983.

ParcPlace Systems Newsletter. Numbers 1–14 are now available from ParcPlace Systems, 2400 Geng Road, Palo Alto, CA 94303.

HOOPLA *(Hooray for Object-Oriented Programming Languages!)*. Newsletter of OOPSTAD (Object-Oriented Programming for Smalltalk Applications Developers Association). Available from OOPSTAD, P.O. Box 1565, Everett, WA 98206.

Further Reading

Stephen T. Pope. Smalltalk-80 Applications Bibliography. *Smalltalk-80 Newsletter #11*. ParcPlace Systems, September 1987.

Trygve Reenskaug. User-Oriented Descriptions of Smalltalk Systems. *Byte*, August, 1981.

Ralph E. Johnson. *Model/View/Controller*. Department of C. S., U. of Illinois, Urbana-Champaign, November 1987.

Sam S. Adams. "MetaMethods: The MVC Paradigm", in: HOOPLA! (Hooray for Object-Oriented Programming Languages!), Vol. 1 Number 4, July, 1988.

To find out more about the use of the MVC classes within the Smalltalk-80 system, interested readers are referred to the system itself. Using the MessageSet browsers for browsing all senders of the pluggable view initialization messages can be very informative. Examples of these might be found in the "plugging" message **on:aspect:change:menu:initialselection:** which is implemented in class CodeView or the parallel messages in the other pluggable view classes such as SelectionInListView or SwitchView.

One can also browse all references to the simple interactive user interface classes (such as FillInTheBlank or BinaryChoice), or the **open** messages for the system's application views. For examples of advanced interaction usage, looking at implementors of the message **controlActivity** can be instructional.